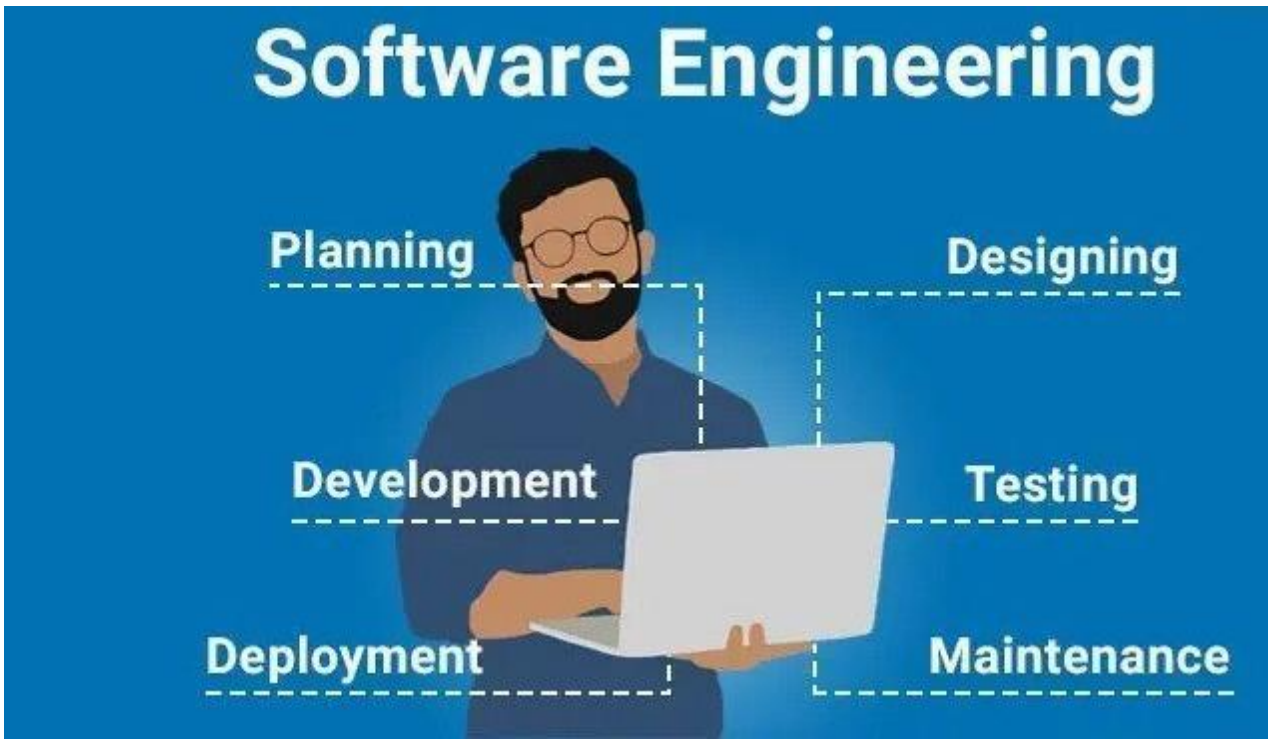
Unit -1 (Introduction to Software Engineering)

1.1 Software Engineering Definition
1.2 Importance of Software Engineering
1.3 Applications of Software Engineering

Software Engineering Definition:-

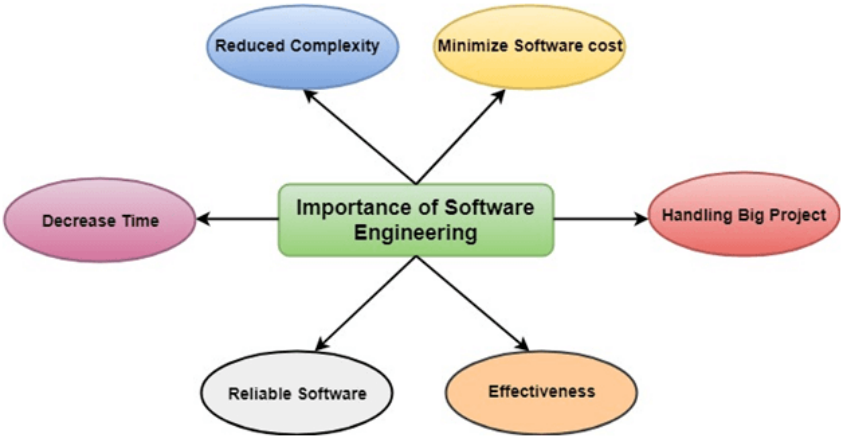
Software is a program or set of programs containing instructions that provide desired functionality. And Engineering is the process of designing and building something that serves a particular purpose and finds a cost-effective solution to problems.



Software Engineering is the process of designing, developing, testing, and maintaining software. It is a systematic and disciplined approach to software development that aims to create high-quality, reliable, and maintainable software. Software engineering includes a variety of techniques, tools, and methodologies, including requirements analysis, design, testing, and maintenance.

Importance of Software Engineering

i) **Handling Large Projects:** Software engineering methodologies are essential for efficiently managing and executing large-scale projects, ensuring that they are completed without major issues or delays.



ii) **Cost Management:** Software engineering enables meticulous planning and the elimination of unnecessary elements, resulting in cost-effective development and reduced project expenses.

iii) **Time Efficiency:** Utilizing software engineering techniques can significantly reduce development time, making it a more time-efficient process.

iv) **Reliability:** Software engineering principles are crucial for delivering software products on time and addressing any defects or issues, enhancing the reliability of the final product.

v) **Effectiveness:** Adhering to industry standards and best practices enhances the effectiveness of software development and ensures a higher level of quality.

vi) **Reducing Complexity:** Software engineering breaks down complex challenges into smaller, manageable components, which are systematically addressed, simplifying the development process.

vii) **Productivity:** Through the incorporation of testing systems at every stage, software engineering ensures the maintenance of software productivity, reducing the likelihood of errors and improving overall efficiency.

Applications of Software Engineering

i) Web Development: Software engineering is crucial for creating websites and web applications, ensuring functionality, security, and user-friendliness. Web developers use software engineering principles to design and maintain online platforms.

ii) Mobile App Development: Mobile app developers use software engineering practices to create applications for smart phones and tablets. This involves designing user interfaces, optimizing performance, and ensuring compatibility across different devices and operating systems.

iii) Embedded Systems: Software engineering plays a critical role in developing software for embedded systems, such as those found in automotive control systems, medical devices, and IoT (Internet of Things) devices.

iv) Game Development: Game developers apply software engineering principles to design and build video games, managing complex graphics, physics, and gameplay mechanics.

v) Financial Software: Financial institutions rely on software engineering to develop software for managing transactions, risk assessment, and customer accounts. This ensures the security and accuracy of financial operations.

vi) Aerospace and Aviation: The aerospace industry uses software engineering for designing control systems, navigation, and communication software in aircraft and spacecraft.

vii) Healthcare Software: Healthcare professionals use software engineering to create electronic health records (EHR) systems, telemedicine platforms, and medical imaging software to improve patient care and streamline medical processes.

viii) E-commerce Platforms: Online retailers depend on software engineering for building and maintaining e-commerce platforms, including shopping carts, payment gateways, and inventory management systems.

ix) Educational Software: Educational institutions use software engineering to develop e-learning platforms, educational games, and academic management systems.

x) Automated Manufacturing: Manufacturing industries use software engineering to implement automation and control systems in production lines, improving efficiency and product quality.

xi) Data Analysis and Big Data: Data scientists and analysts apply software engineering principles when developing data analysis software, including tools for processing and interpreting large datasets.

xii) Cloud Computing: Cloud service providers use software engineering to build and maintain cloud infrastructure, offering scalable and secure computing and storage solutions.

xiii) Social Media Platforms: Social media companies employ software engineering to create and maintain their platforms, managing user interactions, content delivery, and data security.

xiv) Robotics: Robotics engineers use software engineering to program and control robots for various applications, from manufacturing and healthcare to exploration and research.

xv) Operating Systems: Software engineering is fundamental in developing operating systems (e.g., Windows, Linux, macOS) that control computer hardware and provide a platform for running software applications.

xvi) Entertainment and Multimedia: The entertainment industry relies on software engineering for video and audio editing, special effects, animation, and 3D modeling software.

These are just a few examples of the diverse applications of software engineering across various industries. Software engineering principles are essential in ensuring that software and systems are designed, developed, and maintained with high quality, efficiency, and reliability.

Unit - 2 (Project Management Techniques)

2.1 Introduction to Project development techniques
2.2 PERT introduction and implementation
2.3 CPM introduction and implementation
2.4 Implementation of project management techniques in real world

Introduction to Project Development Techniques

Software project development techniques are the processes and practices that software engineers use to plan, design, implement, test, and deploy software products. There are many different project development techniques, each with its own strengths and weaknesses.

Some of the most common project development techniques include:

- **Waterfall:** The waterfall model is a sequential approach to software development. It consists of a series of phases, such as requirements gathering, design, implementation, testing, and deployment. Each phase must be completed before the next phase can begin.
- **Agile:** Agile development is an iterative approach to software development. It involves breaking down the project into smaller chunks of work, called sprints. Each sprint produces a working increment of the software product.
- **DevOps:** DevOps is a set of practices that combines software development and IT operations. It aims to shorten the systems development life cycle and provide continuous delivery with high software quality.

The best project development technique for a particular project will depend on a number of factors, such as the size and complexity of the project, the budget and timeline, and the skills and experience of the development team.

Here are some of the key principles of software project development techniques:

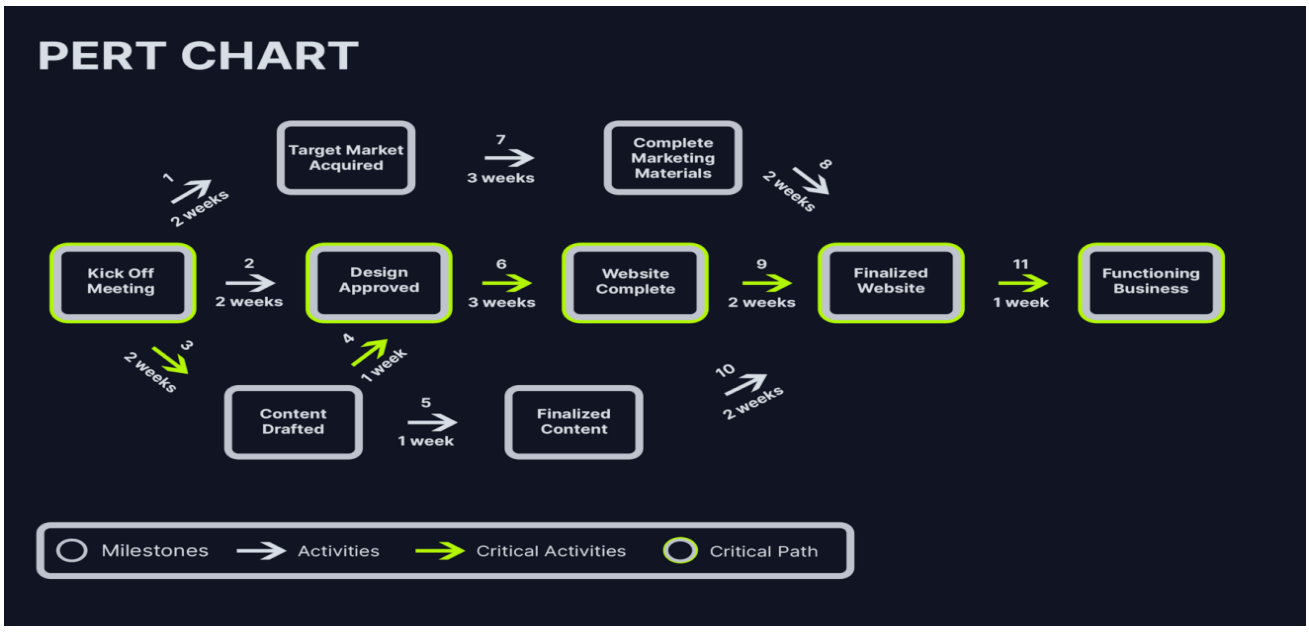
- **Planning:** The first step in any project is to create a plan. This includes defining the project's goals, scope, and schedule. It is also important to identify the resources that will be needed to complete the project.
- **Requirements gathering:** Once the plan is in place, the next step is to gather requirements. This involves working with stakeholders to identify and document the features and functionality that the software product must have.

- **Design:** Once the requirements are gathered, the development team can begin to design the software product. This includes designing the architecture, user interface, and database.
- **Implementation:** Once the design is complete, the development team can begin to implement the software product. This involves writing code, testing the software, and fixing bugs.
- **Testing:** Testing is an essential part of software development. It is important to test the software thoroughly to ensure that it meets the requirements and is free of bugs.
- **Deployment:** Once the software is tested and ready to use, it can be deployed to production. This involves installing the software on the target environment and making it available to users.

By following these principles, software engineers can develop high-quality software products that meet the needs of users and stakeholders.

2.2 PERT introduction and implementation

Project Evaluation and Review Technique (PERT) is a procedure through which activities of a project are represented in its appropriate sequence and timing. It is a scheduling technique used to schedule, organize and integrate tasks within a project. PERT is basically a mechanism for management planning and control which provides blueprint for a particular project. All of the primary elements or events of a project have been finally identified by the PERT.
In this technique, a PERT Chart is made which represent a schedule for all the specified tasks in the project. The reporting levels of the tasks or events in the PERT Charts is somewhat same as defined in the work breakdown structure (WBS).



Characteristics of PERT:

The main characteristics of PERT are as following :-

1. It serves as a base for obtaining the important facts for implementing the decision-making.
2. It forms the basis for all the planning activities.
3. PERT helps management in deciding the best possible resource utilization method.
4. PERT take advantage by using time network analysis technique.
5. PERT presents the structure for reporting information.
6. It helps the management in identifying the essential elements for the completion of the project within time.

Implementation of PERT in Software Engineering:

1. Task Identification: Start by identifying all the tasks required to complete the software project. Tasks can include requirements analysis, design, coding, testing, documentation, and deployment.

2. Dependency Mapping: Determine the dependencies among tasks. For example, coding cannot begin until the design phase is completed, and testing cannot begin until coding is finished. Use arrows to indicate the flow of dependencies.

3. Duration Estimation: Estimate the time required for each task. This involves providing optimistic, pessimistic, and most likely time estimates. These estimates are often based on historical data, expert judgment, and previous project experiences.

4. Network Diagram: Create a network diagram that illustrates the tasks, dependencies, and time estimates. This diagram provides a visual representation of the project's schedule.

5. Critical Path Analysis: Use PERT to calculate the expected duration for each task and identify the critical path. The critical path is the sequence of tasks with the longest expected duration and determines the project's minimum completion time.

6. Resource Allocation: Allocate resources, such as developers, testers, and project managers, to tasks according to the project schedule.

7. Monitoring and Control: Throughout the project, regularly update the PERT chart to reflect actual progress. This allows you to track deviations from the planned schedule and take corrective actions as needed to keep the project on track.

8. Risk Management: PERT helps in identifying potential risks by highlighting critical tasks and dependencies. Effective risk management can help in mitigating potential delays.

Advantages of PERT:

It has the following advantages :

1. Estimation of completion time of project is given by the PERT.
2. It supports the identification of the activities with slack time.
3. The start and dates of the activities of a specific project is determined.
4. It helps project manager in identifying the critical path activities.
5. PERT makes well organized diagram for the representation of large amount of data.

Disadvantages of PERT:

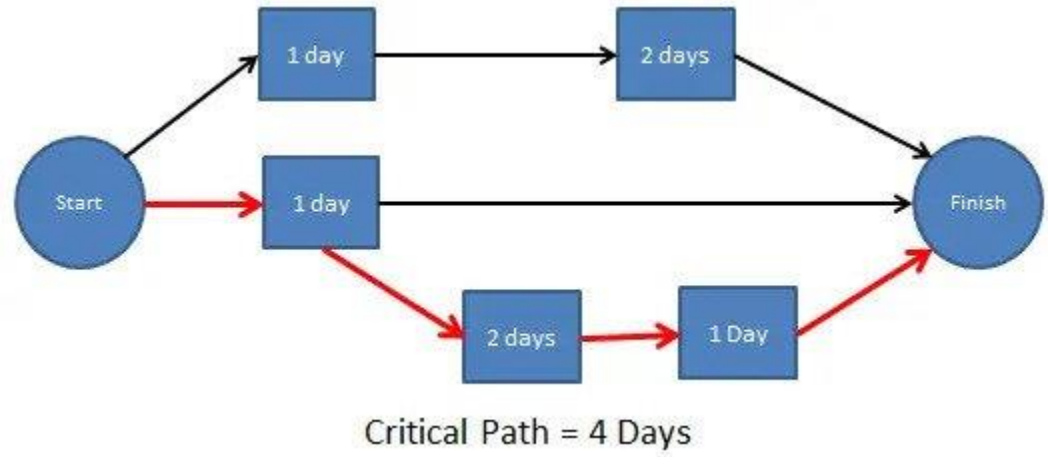
It has the following disadvantages :

1. The complexity of PERT is more which leads to the problem in implementation.
2. The estimation of activity time are subjective in PERT which is a major disadvantage.
3. Maintenance of PERT is also expensive and complex.
4. The actual distribution of may be different from the PERT beta distribution which causes wrong assumptions.
5. It under estimates the expected project completion time as there is chances that other paths can become the critical path if their related activities are deferred.

2.3 CPM introduction and implementation

CPM, or the Critical Path Method, is a project management technique commonly used in software engineering and other industries to plan, schedule, and manage complex projects. CPM focuses on identifying the critical path, which is the

sequence of tasks that must be completed in a specific order to ensure the project is completed as quickly as possible.



Here's an introduction to CPM:

1. Task Identification: In CPM, a project is divided into a set of individual tasks or activities, each of which represents a specific piece of work. These tasks can range from requirements analysis to coding, testing, documentation, and deployment.

2. Dependency Mapping: CPM emphasizes identifying task dependencies. Some tasks can only start after others are finished, and these dependencies are crucial to project scheduling. Dependencies are often represented by arrows between tasks in a network diagram.

3. Task Duration Estimation: Estimating the time required for each task is a critical aspect of CPM. Task duration is usually based on historical data, expert judgment, and experience from previous projects.

4. Network Diagram: Using the task dependencies and duration estimates, a network diagram is created. The nodes in the network diagram represent tasks, and arrows between nodes indicate the task dependencies. The critical path is determined based on the lengths of task sequences in the diagram.

5. Critical Path Identification: The critical path is the longest sequence of dependent tasks, and it defines the minimum amount of time required to complete the entire project. It's important to note that activities on the critical path have zero float or slack, meaning any delay in these tasks will cause a delay in the project's overall completion.

Note:- The above 5 steps or overview are same for both PERT and CPM.

Implementation of CPM in Software Engineering:

1. Task Identification: Begin by identifying all the tasks necessary to complete the software project, from initial concept to deployment. This typically includes requirements analysis, design, coding, testing, documentation, and other project-specific activities.

2. Dependency Mapping: Determine the dependencies between tasks. For example, the design phase must precede the coding phase, and testing cannot begin until the coding phase is complete. Clearly define these dependencies in your project plan.

3. Task Duration Estimation: Estimate the time required for each task. While optimistic, pessimistic, and most likely time estimates, as used in PERT, can be helpful, CPM often relies on single, best-guess estimates. These estimates should be as accurate as possible to avoid scheduling problems.

4. Network Diagram: Create a network diagram that visually represents the tasks, dependencies, and their estimated durations. This diagram helps project managers and teams understand the project's structure and dependencies.

5. Critical Path Identification: Use the network diagram to identify the critical path. The critical path is the longest sequence of dependent tasks, and it determines the project's minimum completion time. Focus on managing and tracking the tasks on the critical path to ensure the project stays on schedule.

6. Resource Allocation: Allocate resources to tasks according to the project schedule. Ensure that team members have a clear understanding of their roles and responsibilities throughout the project.

7. Monitoring and Control: Continuously monitor the progress of the project against the CPM schedule. If any task on the critical path is delayed or if unforeseen issues arise, take corrective actions to bring the project back on track.

8. Risk Management: By identifying the critical path and understanding the dependencies, CPM can help in pinpointing potential risks and areas where project delays might occur. This allows for proactive risk management.

CPM is a valuable project management technique in software engineering that aids in project scheduling, monitoring, and control. By focusing on the critical path, project managers and teams can streamline their efforts, ensure efficient resource allocation, and minimize project duration.

Difference Between PERT and CPM

1. Project Evaluation and Review Technique (PERT) **:**
PERT is appropriate technique which is used for the projects where the time required or needed to complete different activities are not known. PERT is majorly applied for scheduling, organization and integration of different tasks within a project. It provides the blueprint of project and is efficient technique for project evaluation .

2. Critical Path Method (CPM) **:**
CPM is a technique which is used for the projects where the time needed for completion of project is already known. It is majorly used for determining the approximate time within which a project can be completed. Critical path is the largest path in project management which always provide minimum time taken for completion of project

Difference between PERT and CPM :

S.No.	PERT	CPM
1.	PERT is that technique of project management which is used to manage uncertain (i.e., time is not known) activities of any project.	CPM is that technique of project management which is used to manage only certain (i.e., time is known) activities of any project.
2.	It is event oriented technique which means that network is constructed on the basis of event.	It is activity oriented technique which means that network is constructed on the basis of activities.
3.	It is a probability model.	It is a deterministic model.
4.	It majorly focuses on time as meeting time target or estimation of percent completion is more important.	It majorly focuses on Time-cost trade off as minimizing cost is more important.
5.	It is appropriate for high precision time estimation.	It is appropriate for reasonable time estimation.
6.	It has Non-repetitive nature of job.	It has repetitive nature of job.

3.4 Implementation of project management techniques in real world.

Implementation of project management techniques in the real world involves applying a structured and systematic approach to plan, execute, monitor, and control projects. Whether in software development, construction, healthcare, or any other industry, effective project management is crucial for successful project outcomes. Here's a general overview of how project management techniques are implemented in the real world:

1. **Project Initiation:**

- Identify the need for a project.
- Define the project's objectives, scope, and constraints.
- Appoint a project manager and assemble a project team.

2. **Project Planning:**

- Develop a project plan outlining tasks, resources, timeline, and budget.
- Define the project's scope and objectives clearly.
- Identify stakeholders and establish communication channels.
- Create a risk management plan to identify and mitigate potential challenges.
- Choose the appropriate project management methodology (e.g., Agile, Waterfall, Scrum, etc.).

3. **Task and Resource Allocation:**

- Assign responsibilities and tasks to team members.
- Allocate necessary resources, including human resources, equipment, and materials.
- Ensure that roles and responsibilities are well-defined.

4. **Scheduling:**

    - Develop a project schedule that outlines task sequences and timelines.
    - Use tools like Gantt charts or project management software to visualize the schedule.
    - Define dependencies between tasks and identify the critical path.

5. **Execution:**

- Start the actual work according to the project plan.
- Monitor progress, ensuring tasks are completed on time and within budget.
- Manage risks and unforeseen issues as they arise.
- Maintain clear communication within the project team and with stakeholders.

6. Monitoring and Control:

- Continuously track the project's progress against the schedule and budget.
- Implement change management processes for handling scope changes or deviations.
- Address any issues or roadblocks promptly.
- Use key performance indicators (KPIs) to measure progress and quality.

7. Quality Assurance:

- Implement quality control measures to ensure that project deliverables meet predefined standards.
- Perform testing, reviews, and inspections to identify and address defects.

8. Communication:

- Maintain open and transparent communication among team members, stakeholders, and relevant parties.
- Regularly update stakeholders on project progress, issues, and changes.

9. Risk Management:

- Continuously assess and address project risks.
- Adjust the risk management plan as needed to mitigate potential challenges.

10. Documentation:

- Keep detailed records of project activities, decisions, and changes.
- Document lessons learned for future reference.

11. Closure and Evaluation:

- Ensure that all project objectives have been met.

- Conduct a project review to evaluate the project's success and areas for improvement.
- Obtain formal acceptance of the project's deliverables from stakeholders.
- Transition the project to ongoing operations, if applicable.

12. Lessons Learned:

- After the project is completed, analyze what went well and what could be improved.
- Document lessons learned to inform future projects.

13. Continuous Improvement:

- Use insights from past projects to improve project management techniques and processes for future endeavors.

The implementation of project management techniques in the real world involves a combination of methodology, communication, and tools. Successful project management requires adaptability and the ability to tailor techniques to the specific needs and characteristics of each project. Additionally, the use of project management software and technology can greatly enhance efficiency and collaboration in today's increasingly complex projects.

Unit - 3(Software Development Phases)

3.1 Importance and need of SDLC
3.2 System Study
3.3 Feasibility study and its types
3.4 System Requirements & Analysis
3.5 System Requirements Specification (SRS)
3.6 System Design
3.7 System Development
3.8 System Testing
3.9 System Implementation
3.10 System Maintenance and Reviews

Importance and need of SDLC

Software Development Life Cycle (SDLC)

A software life cycle model (also termed process model) is a pictorial and diagrammatic representation of the software life cycle. A life cycle model represents all the methods required to make a software product transit through its life cycle stages. It also captures the structure in which these methods are to be undertaken.

In other words, a life cycle model maps the various activities performed on a software product from its inception to retirement. Different life cycle models may plan the necessary development activities to phases in different ways. Thus, no element which life cycle model is followed, the essential activities are contained in all life cycle models though the action may be carried out in distinct orders in different life cycle models. During any life cycle stage, more than one activity may also be carried out.

The SDLC is important for a number of reasons:

- It helps to ensure the quality of the software. The SDLC includes a number of quality assurance activities, such as testing and code review. These activities help to identify and fix bugs early in the development process, which can save time and money in the long run.
- It helps to reduce the risk of project failure. The SDLC helps software teams to identify and mitigate risks throughout the development process. This helps to reduce the chances of the project failing.

- It helps to improve communication and collaboration. The SDLC provides a framework for communication and collaboration between different stakeholders in the development process, such as software engineers, business analysts, and quality assurance engineers. This helps to ensure that everyone is on the same page and that the project is progressing smoothly.
- It helps to manage costs and schedule. The SDLC helps software teams to estimate the costs and schedule for the project. This helps to ensure that the project is completed on time and within budget.

The need for SDLC arises from the fact that software development is a complex process with many moving parts. Without a structured process in place, it is easy for things to go wrong. The SDLC helps to reduce this complexity and to ensure that the project is completed successfully.

Here are some specific examples of how the SDLC can be used to improve the software development process:

- The SDLC can help to ensure that the software meets the needs of the users. By gathering and analyzing user requirements early in the development process, the SDLC helps to ensure that the software is designed and implemented to meet those requirements.
- The SDLC can help to identify and fix bugs early in the development process. By testing the software regularly throughout the development process, the SDLC helps to identify and fix bugs before they become serious problems.
- The SDLC can help to improve the maintainability of the software. By designing the software with maintainability in mind, the SDLC helps to make the software easier to update and fix in the future.
- The SDLC can help to reduce the cost of developing and maintaining the software. By following a structured process, the SDLC helps to avoid costly mistakes and rework.

Need of SDLC

The development team must determine a suitable life cycle model for a particular plan and then observe to it.

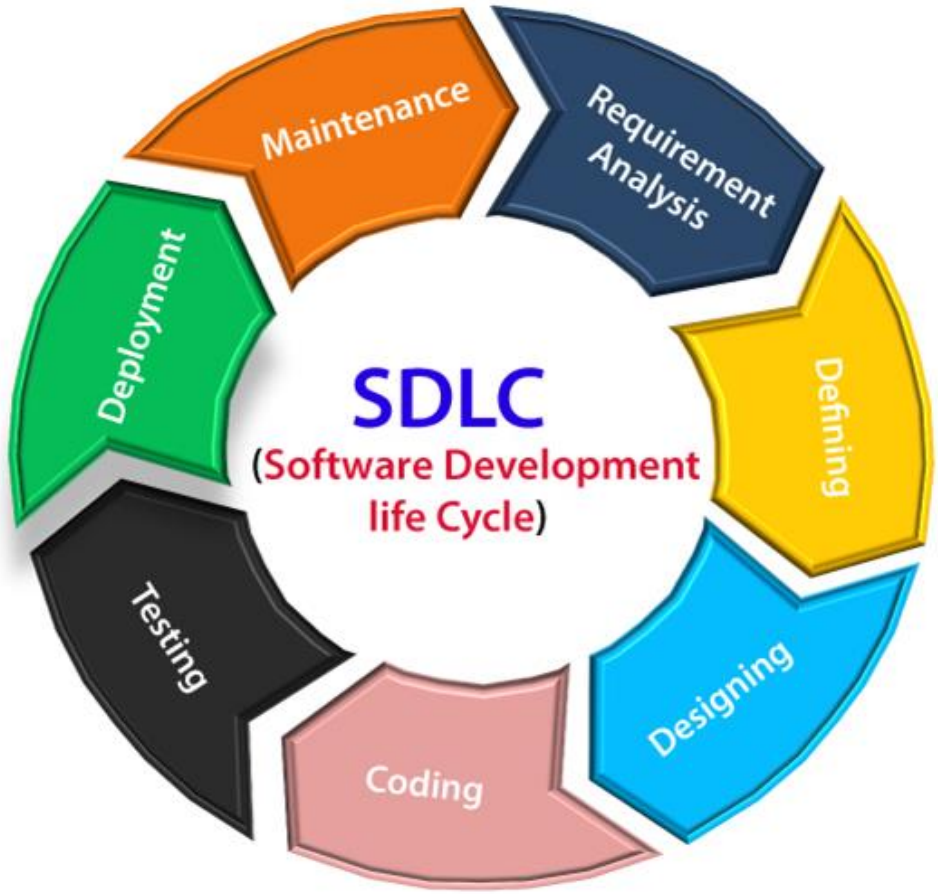
Without using an exact life cycle model, the development of a software product would not be in a systematic and disciplined manner. When a team is developing a software product, there must be a clear understanding among team representative about when and what to do. Otherwise, it would point to chaos and project failure. This problem can be defined by using an example. Suppose a

software development issue is divided into various parts and the parts are assigned to the team members. From then on, suppose the team representative is allowed the freedom to develop the roles assigned to them in whatever way they like. It is possible that one representative might start writing the code for his part, another might choose to prepare the test documents first, and some other engineer might begin with the design phase of the roles assigned to him. This would be one of the perfect methods for project failure.

A software life cycle model describes entry and exit criteria for each phase. A phase can begin only if its stage-entry criteria have been fulfilled. So without a software life cycle model, the entry and exit criteria for a stage cannot be recognized. Without software life cycle models, it becomes tough for software project managers to monitor the progress of the project.

SDLC Cycle

SDLC Cycle represents the process of developing software. SDLC framework includes the following steps:



The stages of SDLC are as follows:

- Phase 1: Requirement collection and analysis
- Phase 2: Feasibility study
- Phase 3: Design
- Phase 4: Coding
- Phase 5: Testing
- Phase 6: Installation/Deployment
- Phase 7: Maintenance

Phase 1: Requirement collection and analysis or System Study

The requirement is the first stage in the SDLC process. It is conducted by the senior team members with inputs from all the stakeholders and domain experts in the industry. Planning for the quality assurance requirements and recognization of the risks involved is also done at this stage.

This stage gives a clearer picture of the scope of the entire project and the anticipated issues, opportunities, and directives which triggered the project.

Requirements Gathering stage need teams to get detailed and precise requirements. This helps companies to finalize the necessary timeline to finish the work of that system.

Phase 2: Feasibility study and it's types

Once the requirement analysis phase is completed the next sdlc step is to define and document software needs. This process conducted with the help of ‘Software Requirement Specification’ document also known as ‘SRS’ document. It includes everything which should be designed and developed during the project life cycle.

There are mainly five types of feasibilities checks:

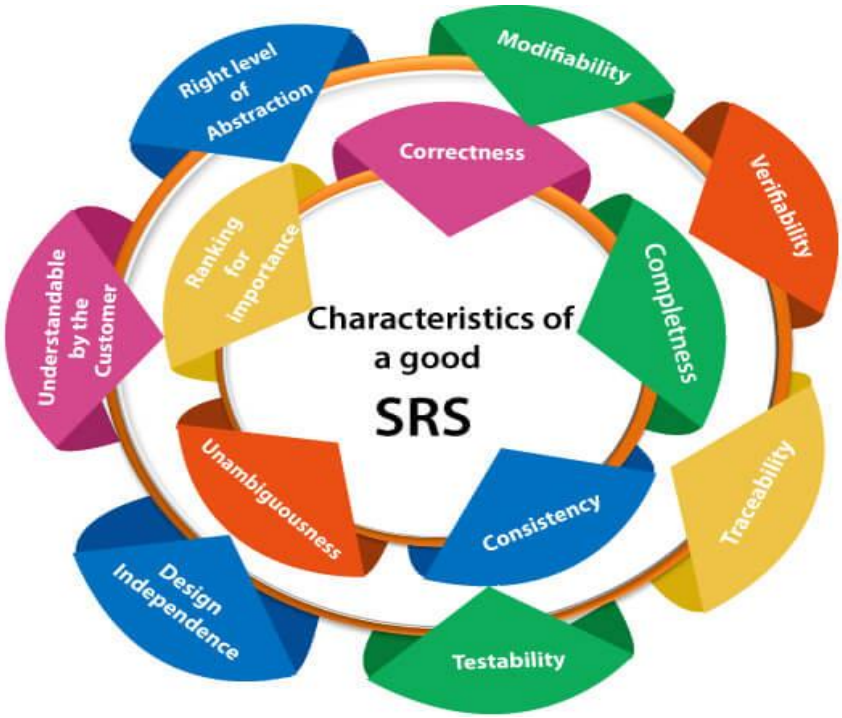
- **Economic:** Can we complete the project within the budget or not?
- **Legal:** Can we handle this project as cyber law and other regulatory framework/compliances.
- **Operation feasibility:** Can we create operations which is expected by the client?
- **Technical:** Need to check whether the current computer system can support the software
- **Schedule:** Decide that the project can be completed within the given schedule or not.

System Requirements Specification(SRS)

The production of the requirements stage of the software development process is **Software Requirements Specifications (SRS)** (also called a **requirements document**). This report lays a foundation for software engineering activities and is constructing when entire requirements are elicited and analyzed. **SRS** is a formal report, which acts as a representation of software that enables the customers to review whether it (SRS) is according to their requirements. Also, it comprises user requirements for a system as well as detailed specifications of the system requirements.

The SRS is a specification for a specific software product, program, or set of applications that perform particular functions in a specific environment. It serves several goals depending on who is writing it. First, the SRS could be written by the client of a system. Second, the SRS could be written by a developer of the system. The two methods create entirely various situations and establish different purposes for the document altogether. The first case, SRS, is used to define the needs and expectation of the users. The second case, SRS, is written for various purposes and serves as a contract document between customer and developer.

Characteristics of good SRS



1. **Correctness:** SRS should be accurate and cover all expected system needs.
2. **Completeness:** It should include all essential requirements, responses to input data, labels, references, and terms.
3. **Consistency:** No conflicting requirements, including characteristics, actions, and terms.
4. **Unambiguousness:** Each requirement has a single interpretation, avoiding multiple definitions.
5. **Ranking:** Requirements are ranked for importance and stability.
6. **Modifiability:** SRS should be easily modifiable and well-documented.
7. **Verifiability:** Requirements are checked for correctness through reviews.
8. **Traceability:** Clear origin and referencing of requirements in backward and forward traceability.
9. **Design Independence:** No implementation details in SRS, allowing for multiple design alternatives.
10. **Testability:** SRS should enable the generation of test cases and plans.
11. **Understandable by the customer:** Use simple language, avoid formal notations, and keep it clear.
12. **Right Level of Abstraction:** The level of detail varies according to the SRS's purpose, from requirements stage to feasibility study.

Uses of SRS document

- Development team require it for developing product according to the need.
- Test plans are generated by testing group based on the describe external behaviour.
- Maintenance and support staff need it to understand what the software product is supposed to do.
- Project manager base their plans and estimates of schedule, effort and resources on it.
- customer rely on it to know that product they can expect.
- As a contract between developer and customer.
- In documentation purpose.

Phase 3: Design

In this third phase, the system and software design documents are prepared as per the requirement specification document. This helps define overall system architecture.

This design phase serves as input for the next phase of the model.

There are two kinds of design documents developed in this phase:

High-Level Design (HLD)

- Brief description and name of each module
- An outline about the functionality of every module
- Interface relationship and dependencies between modules
- Database tables identified along with their key elements
- Complete architecture diagrams along with technology details

Low-Level Design (LLD)

- Functional logic of the modules
- Database tables, which include type and size
- Complete detail of the interface
- Addresses all types of dependency issues
- Listing of error messages
- Complete input and outputs for every module

Phase 4: Coding or Development

Once the system design phase is over, the next phase is coding. In this phase, developers start build the entire system by writing code using the chosen programming language. In the coding phase, tasks are divided into units or modules and assigned to the various developers. It is the longest phase of the Software Development Life Cycle process.

In this phase, Developer needs to follow certain predefined coding guidelines. They also need to use programming tools like compiler, interpreters, debugger to generate and implement the code.

Phase 5: Testing

Once the software is complete, and it is deployed in the testing environment. The testing team starts testing the functionality of the entire system. This is done to verify that the entire application works according to the customer requirement.

During this phase, QA and testing team may find some bugs/defects which they communicate to developers. The development team fixes the bug and send back to QA for a re-test. This process continues until the software is bug-free, stable, and working according to the business needs of that system.

Phase 6: Installation/Deployment / Implementation

Once the software testing phase is over and no bugs or errors left in the system then the final deployment process starts. Based on the feedback given by the project manager, the final software is released and checked for deployment issues if any.

Phase 7: Maintenance and Reviews

Once the system is deployed, and customers start using the developed system, following 3 activities occur

- Bug fixing – bugs are reported because of some scenarios which are not tested at all
- Upgrade – Upgrading the application to the newer versions of the Software
- Enhancement – Adding some new features into the existing software

The main focus of this SDLC phase is to ensure that needs continue to be met and that the system continues to perform as per the specification mentioned in the first phase.

Unit - 4 (Software Development life Cycle Models)

4.1 Waterfall Model
4.2 Prototyping Model
4.3 Spiral Model
4.4 RAD Model

Software Development Life Cycle (SDLC) models are structured frameworks that guide the development of software applications. They provide a systematic approach to software development, outlining phases, activities, and the sequence in which they should be executed. Different models suit different project requirements and constraints.

Waterfall Model

The Waterfall Model is a linear sequential flow. In which progress is seen as flowing steadily downwards (like a waterfall) through the phases of software implementation. This means that any phase in the development process begins only if the previous phase is complete. The waterfall approach does not define the process to go back to the previous phase to handle changes in requirement. The waterfall approach is the earliest approach and most widely known that was used for software development.

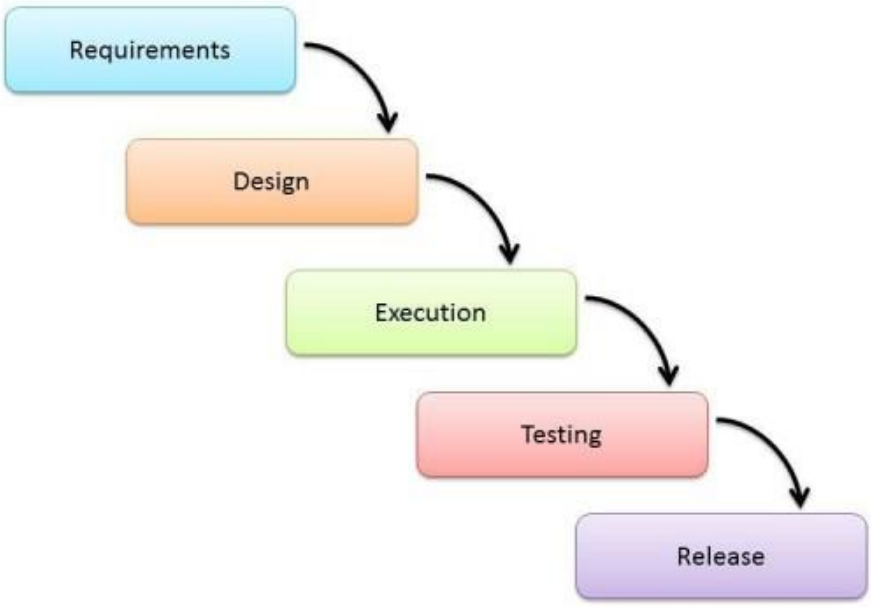


Fig :- Waterfall Model

The usage

For projects which not focus on changing the requirements, for example, projects initiated from a request for proposals (RFPs), the customer has very clear documented requirements

Advantages and Disadvantages

Advantages	Disadvantages
Easy to explain to the users. Structures approach. Stages and activities are well defined. Helps to plan and schedule the project. Verification at each stage ensures early detection of errors/misunderstandings. Each phase has specific deliverables.	Assumes that the requirements of a system can be frozen. Very difficult to go back to any stage after it is finished. A little flexibility and adjusting the scope are difficult and expensive. Costly and required more time, in addition to the detailed plan.

Prototyping Model

It refers to the activity of creating prototypes of software applications, for example, incomplete versions of the software program being developed. It is an activity that can occur in software development and It used to visualize some components of the software to limit the gap of misunderstanding the customer requirements by the development team. This also will reduce the iterations that may occur in the waterfall approach and are hard to be implemented due to the inflexibility of the waterfall approach. So, when the final prototype is developed, the requirement is considered to be frozen.

It has some types, such as:

- Throwaway prototyping: Prototypes that are eventually discarded rather than becoming a part of the finally delivered software

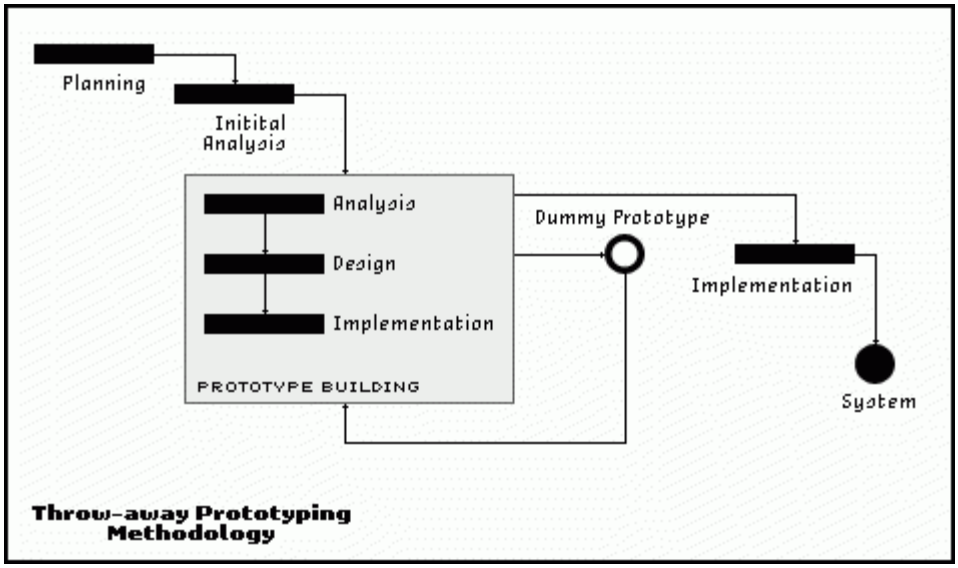


Fig :- Throwaway prototyping

- Evolutionary prototyping: prototypes that evolve into the final system through an iterative incorporation of user feedback.

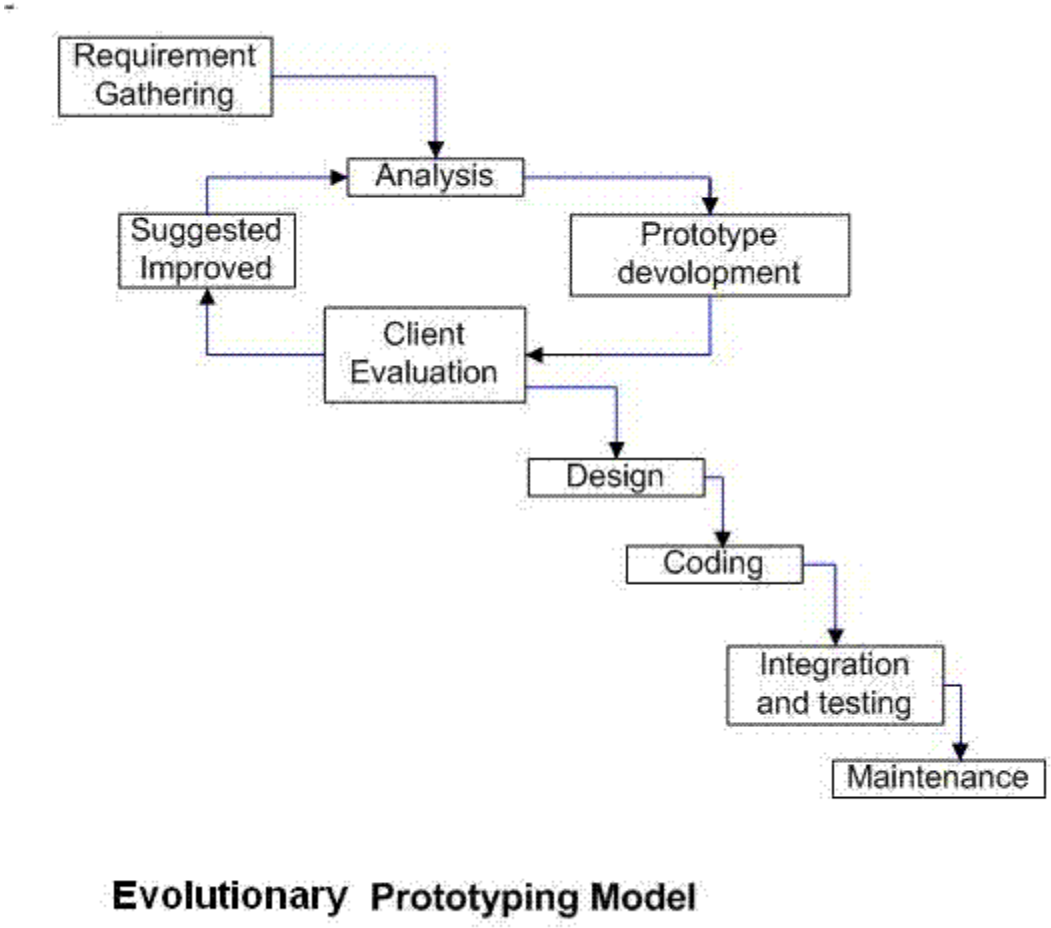


Fig :- Evolutionary prototyping

- Incremental prototyping: The final product is built as separate prototypes. In the end, the separate prototypes are merged in an overall design.

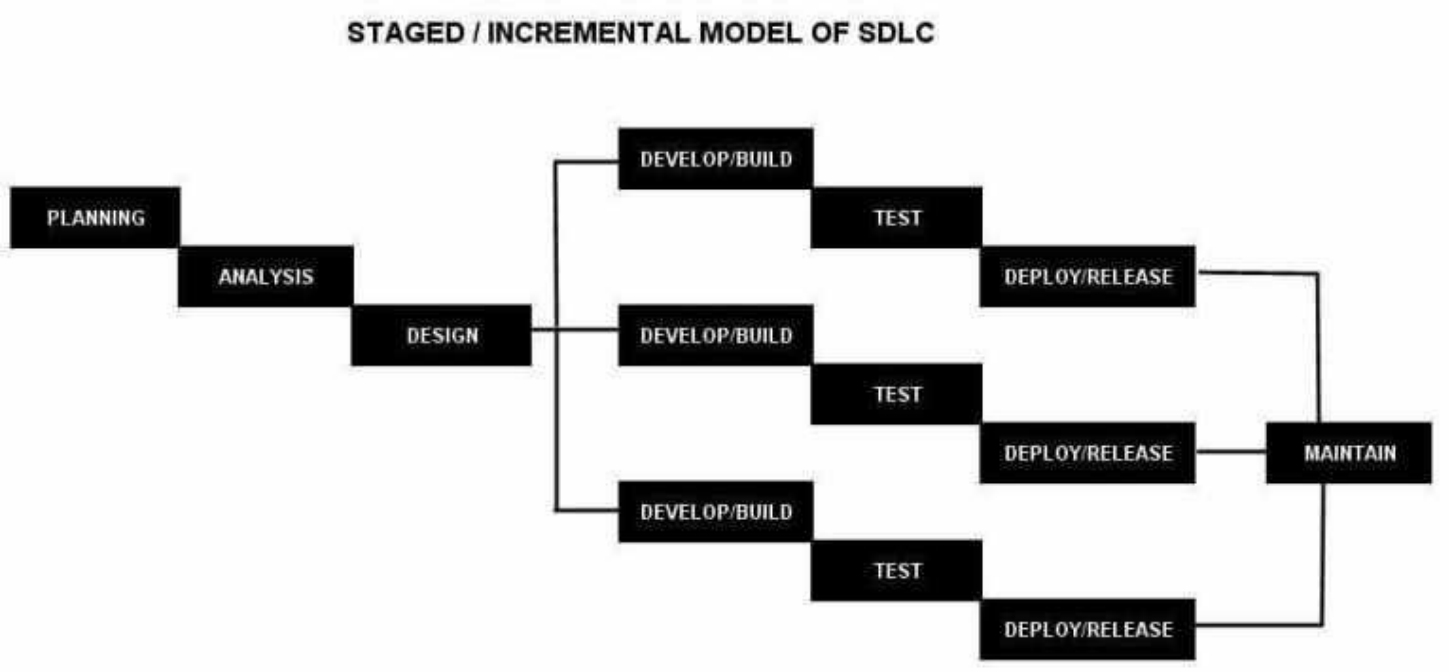


Fig :- Incremental prototyping

- Extreme prototyping: used in web applications mainly. Basically, it breaks down web development into three phases, each one based on the preceding one. The first phase is a static prototype that consists mainly of HTML pages. In the second phase, the screens are programmed and fully functional using a simulated services layer. In the third phase, the services are implemented

The usage

- This process can be used with any software developing life cycle model. While this shall be chosen when you are developing a system has user interactions. So, if the system does not have user interactions, such as a system does some calculations shall not have prototypes.

Advantages and Disadvantages

Advantages	Disadvantages
◦ Reduced time and costs, but this can be a disadvantage if the developer loses time in developing the prototypes. ◦ Improved and increased user involvement.	◦ Insufficient analysis. User confusion of prototype and finished system. ◦ Developer misunderstanding of user objectives. ◦ Excessive development time of the prototype. ◦ It is costly to implement the prototypes

Spiral Model (SDM)

It is combining elements of both design and prototyping-in-stages, in an effort to combine advantages of top-down and bottom-up concepts. This model of development combines the features of the prototyping model and the waterfall model. The spiral model is favored for large, expensive, and complicated projects. This model uses many of the same phases as the waterfall model, in essentially the same order, separated by planning, risk assessment, and the building of prototypes and simulations.

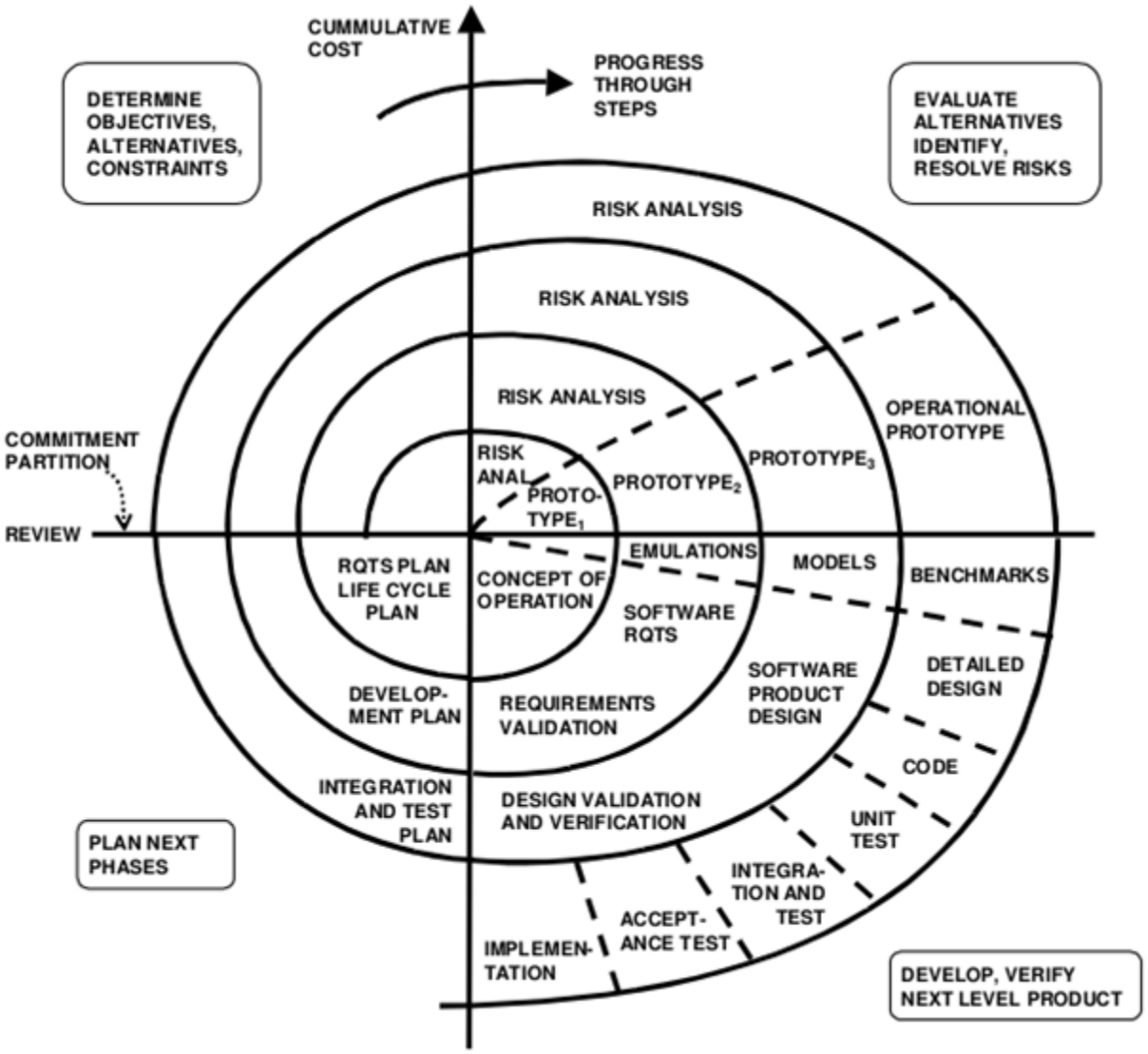


Fig:- Spiral Model

The usage

It is used in large applications and systems which built-in small phases or segments.

Advantages and Disadvantages

Advantages	Disadvantages
• Estimates (i.e. budget, schedule, etc.) become more realistic as work progressed because important issues are discovered earlier. • Early involvement of developers. • Manages risks and develops the system into phases.	• High cost and time to reach the final product. • Needs special skills to evaluate the risks and assumptions. • Highly customized limiting re-usability

RAD Model

RAD or Rapid Application Development process is an adoption of the waterfall model; it targets developing software in a short period. The RAD model is based on the concept that a better system can be developed in lesser time by using focus groups to gather system requirements.

- Business Modeling
- Data Modeling
- Process Modeling
- Application Generation
- Testing and Turnover

Advantages of the RAD model:

- It is fast and efficient.
- It delivers working software quickly.
- It is adaptable to change.

Disadvantages of the RAD model:

- It requires a high level of expertise and experience from the development team.
- It is not suitable for complex projects.
- It can be difficult to manage large projects.

Which SDLC model is best?

The best SDLC model for a particular project will depend on a number of factors, such as the size and complexity of the project, the budget and timeline, and the skills and experience of the development team.

Unit - 5 (Software Analysis and Design Tools)

5.1 Dataflow diagram (DFD), ER Diagram
5.2 Structure Chart
5.3 Decision Table
5.4 Decision Tree
5.5 Use case Diagram
5.6 Sequence Diagram

Dataflow diagram and ER diagram

DFD is the abbreviation for **Data Flow Diagram**. The flow of data of a system or a process is represented by DFD. It also gives insight into the inputs and outputs of each entity and the process itself. DFD does not have control flow and no loops or decision rules are present. Specific operations depending on the type of data can be explained by a flowchart. It is a graphical tool, useful for communicating with users ,managers and other personnel. it is useful for analyzing existing as well as proposed system.

It should be pointed out that a DFD is not a flowchart. In drawing the DFD, the designer has to specify the major transforms in the path of the data flowing from the input to the output. DFDs can be hierarchically organized, which helps in progressively partitioning and analyzing large systems.

It provides an overview of

- What data is system processes.
- What transformation are performed.
- What data are stored.
- What results are produced , etc.

Data Flow Diagram can be represented in several ways. The DFD belongs to structured-analysis modeling tools. Data Flow diagrams are very popular because they help us to visualize the major steps and data involved in software-system processes.

Characteristics of DFD

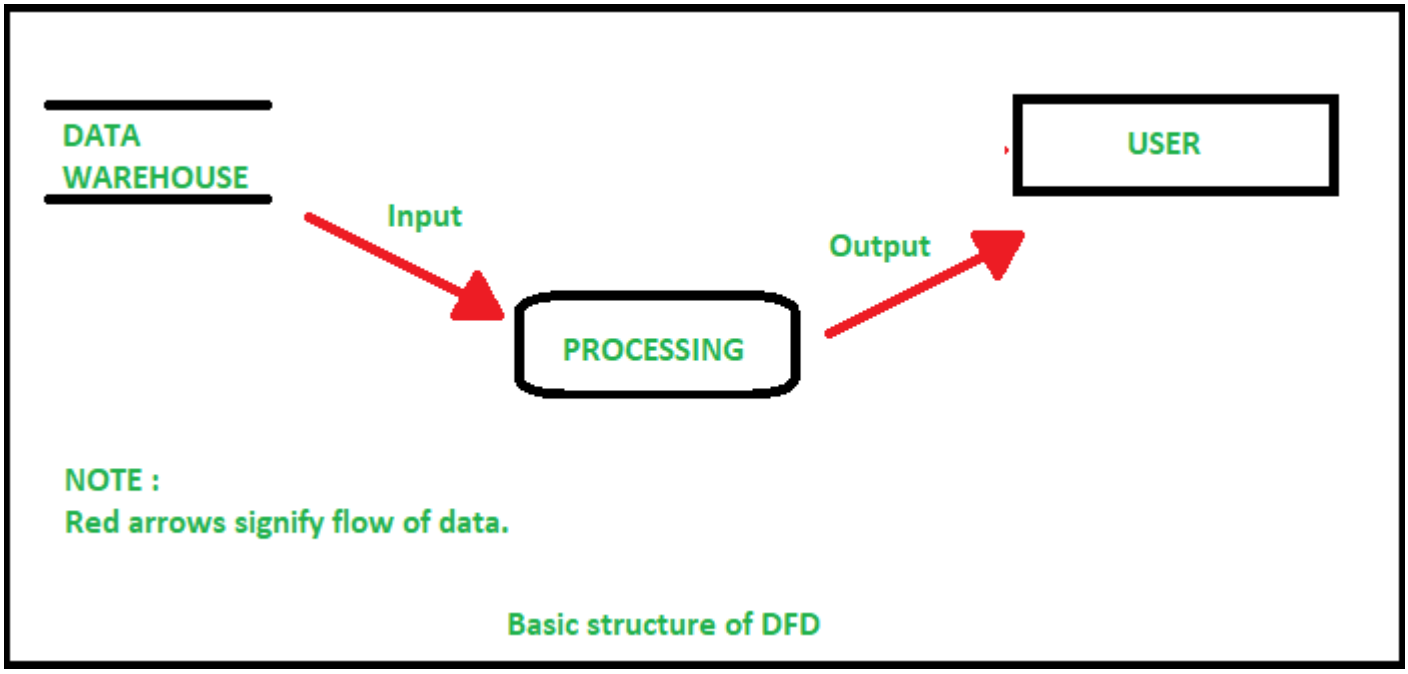
- DFDs are commonly used during problem analysis.
- DFDs are quite general and are not limited to problem analysis for software requirements specification.

- DFDs are very useful in understanding a system and can be effectively used during analysis.
- It views a system as a function that transforms the inputs into desired outputs.
- The DFD aims to capture the transformations that take place within a system to the input data so that eventually the output data is produced.
- The processes are shown by named circles and data flows are represented by named arrows entering or leaving the bubbles.
- A rectangle represents a source or sink and it is a net originator or consumer of data. A source sink is typically outside the main system of study.

Components of DFD

The Data Flow Diagram has 4 components:

- **Process** Input to output transformation in a system takes place because of process function. The symbols of a process are rectangular with rounded corners, oval, rectangle or a circle. The process is named a short sentence, in one word or a phrase to express its essence
- **Data Flow** Data flow describes the information transferring between different parts of the systems. The arrow symbol is the symbol of data flow. A relatable name should be given to the flow to determine the information which is being moved. Data flow also represents material along with information that is being moved. Material shifts are modeled in systems that are not merely informative. A given flow should only transfer a single type of information. The direction of flow is represented by the arrow which can also be bi-directional.
- **Warehouse** The data is stored in the warehouse for later use. Two horizontal lines represent the symbol of the store. The warehouse is simply not restricted to being a data file rather it can be anything like a folder with documents, an optical disc, a filing cabinet. The data warehouse can be viewed independent of its implementation. When the data flow from the warehouse it is considered as data reading and when data flows to the warehouse it is called data entry or data updating.
- **Terminator** The Terminator is an external entity that stands outside of the system and communicates with the system. It can be, for example, organizations like banks, groups of people like customers or different departments of the same organization, which is not a part of the model system and is an external entity. Modeled systems also communicate with terminator.



Basic structure of DFD

Rules for creating DFD

- The name of the entity should be easy and understandable without any extra assistance(like comments).
- The processes should be numbered or put in ordered list to be referred easily.
- The DFD should maintain consistency across all the DFD levels.
- A single DFD can have a maximum of nine processes and a minimum of three processes.

Symbols Used in DFD

- **Square Box:** A square box defines source or destination of the system. It is also called entity. It is represented by rectangle.
- **Arrow or Line:** An arrow identifies the data flow i.e. it gives information to the data that is in motion.
- **Circle or bubble chart:** It represents as a process that gives us information. It is also called processing box.
- **Open Rectangle:** An open rectangle is a data store. In this data is store either temporary or permanently.

Levels of DFD

DFD uses hierarchy to maintain transparency thus multilevel DFD's can be created. Levels of DFD are as follows:

- 0-level DFD: It represents the entire system as a single bubble and provides an overall picture of the system.
- 1-level DFD: It represents the main functions of the system and how they interact with each other.
- 2-level DFD: It represents the processes within each function of the system and how they interact with each other.

- 3-level DFD: It represents the data flow within each process and how the data is transformed and stored.

Advantages of DFD

- It helps us to understand the functioning and the limits of a system.
- It is a graphical representation which is very easy to understand as it helps visualize contents.
- Data Flow Diagram represent detailed and well explained diagram of system components.
- It is used as the part of system documentation file.
- Data Flow Diagrams can be understood by both technical or nontechnical person because they are very easy to understand.

Disadvantages of DFD

- At times DFD can confuse the programmers regarding the system.
- Data Flow Diagram takes long time to be generated, and many times due to this reasons analysts are denied permission to work on it.

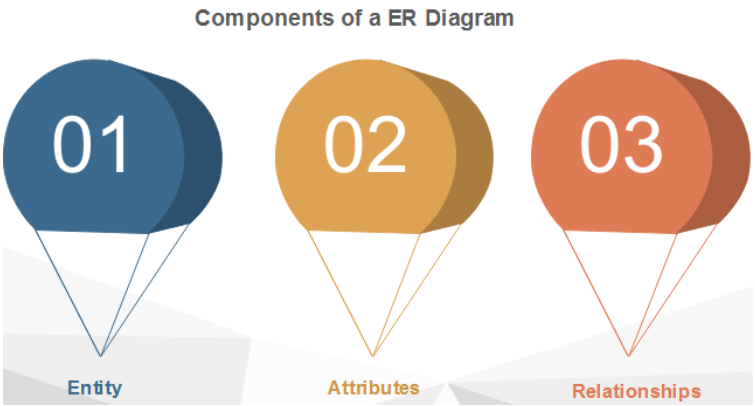
Entity-Relationship Diagrams

ER-modeling is a data modeling method used in software engineering to produce a conceptual data model of an information system. Diagrams created using this ER-modeling method are called Entity-Relationship Diagrams or ER diagrams or ERDs.

Purpose of ERD

- The database analyst gains a better understanding of the data to be contained in the database through the step of constructing the ERD.
- The ERD serves as a documentation tool.
- Finally, the ERD is used to connect the logical structure of the database to users. In particular, the ERD effectively communicates the logic of the database to users.

Components of an ER Diagrams



1. Entity

An entity can be a real-world object, either animate or inanimate, that can be merely identifiable. An entity is denoted as a rectangle in an ER diagram. For example, in a school database, students, teachers, classes, and courses offered can be treated as entities. All these entities have some attributes or properties that give them their identity.

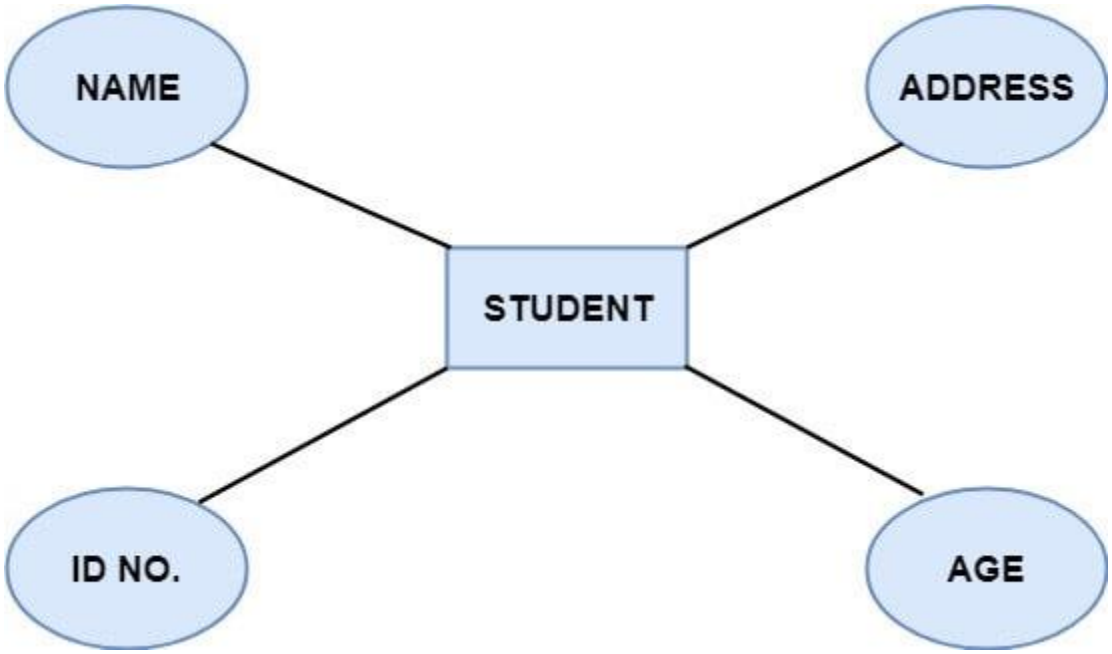
Entity Set

An entity set is a collection of related types of entities. An entity set may include entities with attribute sharing similar values. For example, a Student set may contain all the students of a school; likewise, a Teacher set may include all the teachers of a school from all faculties. Entity set need not be disjoint.

2. Attributes

Entities are denoted utilizing their properties, known as attributes. All attributes have values. For example, a student entity may have name, class, and age as attributes.

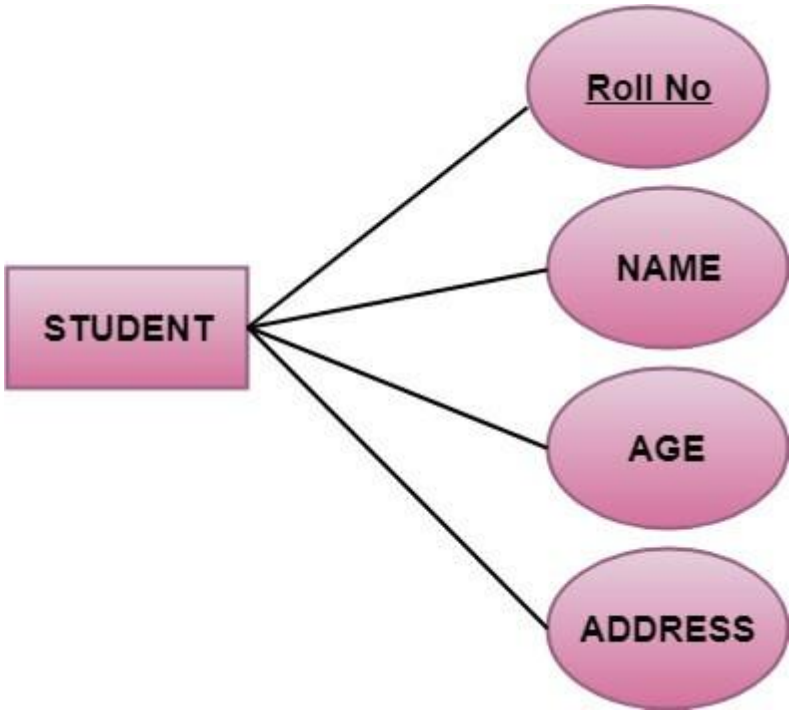
There exists a domain or range of values that can be assigned to attributes. For example, a student's name cannot be a numeric value. It has to be alphabetic. A student's age cannot be negative, etc.



There are four types of Attributes:

1. Key attribute
2. Composite attribute
3. Single-valued attribute
4. Multi-valued attribute
5. Derived attribute

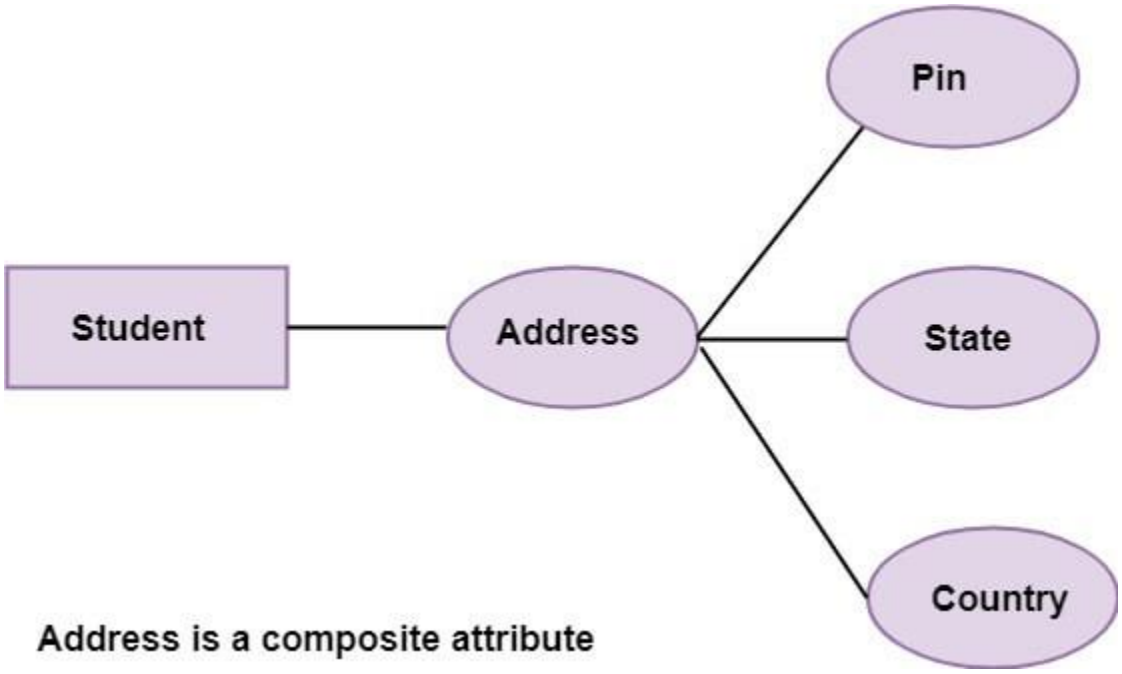
1. Key attribute: Key is an attribute or collection of attributes that uniquely identifies an entity among the entity set. For example, the roll_number of a student makes him identifiable among students.



There are mainly three types of keys:

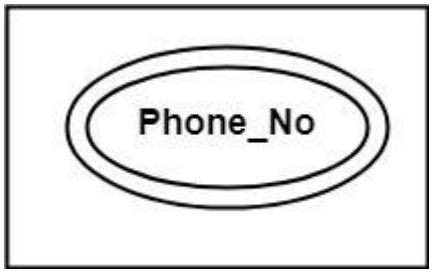
1. **Super key:** A set of attributes that collectively identifies an entity in the entity set.
2. **Candidate key:** A minimal super key is known as a candidate key. An entity set may have more than one candidate key.
3. **Primary key:** A primary key is one of the candidate keys chosen by the database designer to uniquely identify the entity set.

2. Composite attribute: An attribute that is a combination of other attributes is called a composite attribute. For example, In student entity, the student address is a composite attribute as an address is composed of other characteristics such as pin code, state, country.

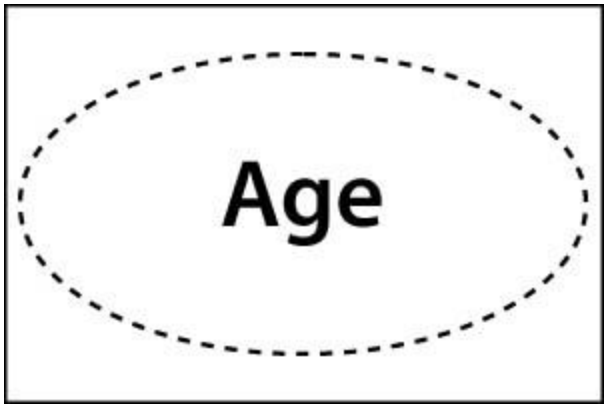


3. Single-valued attribute: Single-valued attribute contain a single value. For example, Social_Security_Number.

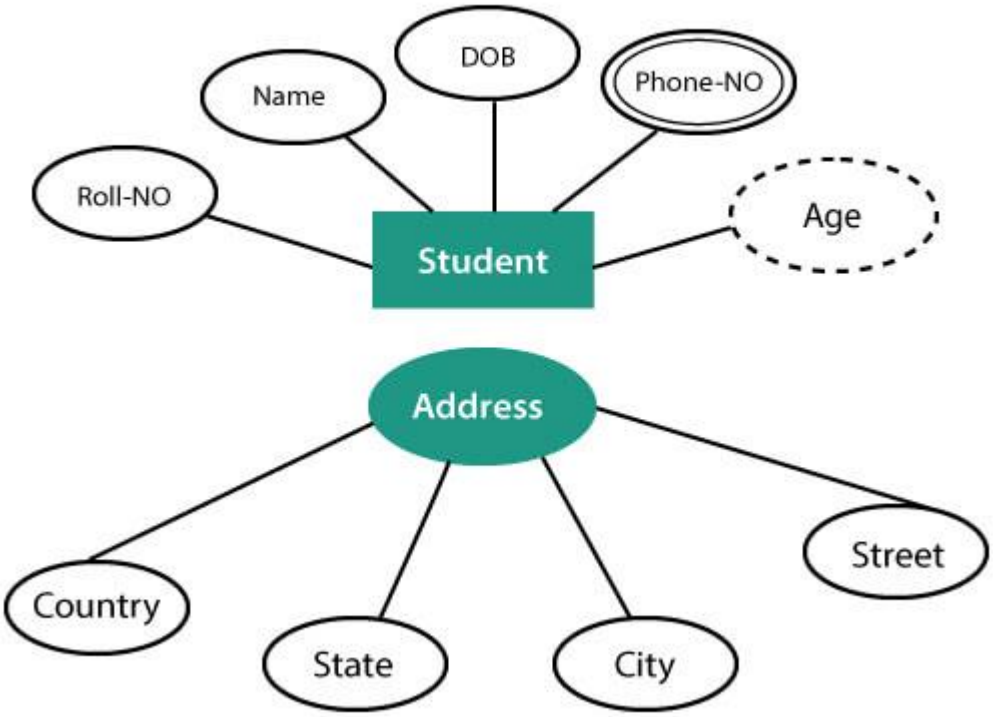
4. Multi-valued Attribute: If an attribute can have more than one value, it is known as a multi-valued attribute. Multi-valued attributes are depicted by the double ellipse. For example, a person can have more than one phone number, email-address, etc.



5. Derived attribute: Derived attributes are the attribute that does not exist in the physical database, but their values are derived from other attributes present in the database. For example, age can be derived from date_of_birth. In the ER diagram, Derived attributes are depicted by the dashed ellipse.



The Complete entity type Student with its attributes can be represented as:



3. Relationships

The association among entities is known as relationship. Relationships are represented by the diamond-shaped box. For example, an employee works_at a department, a student enrolls in a course. Here, Works_at and Enrolls are called relationships.

Fig: Relationships in ERD

Relationship set

A set of relationships of a similar type is known as a relationship set. Like entities, a relationship too can have attributes. These attributes are called descriptive attributes.

Degree of a relationship set

The number of participating entities in a relationship describes the degree of the relationship. The three most common relationships in E-R models are:

1. Unary (degree1)
2. Binary (degree2)
3. Ternary (degree3)

1. Unary relationship: This is also called recursive relationships. It is a relationship between the instances of one entity type. For example, one person is married to only one person.

Fig: Unary Relationship

2. Binary relationship: It is a relationship between the instances of two entity types. For example, the Teacher teaches the subject.

Fig: Binary Relationship

3. Ternary relationship: It is a relationship amongst instances of three entity types. In fig, the relationships "**may have**" provide the association of three entities, i.e., TEACHER, STUDENT, and SUBJECT. All three entities are many-to-many participants. There may be one or many participants in a ternary relationship.

In general, "**n**" entities can be related by the same relationship and is known as **n-ary** relationship.

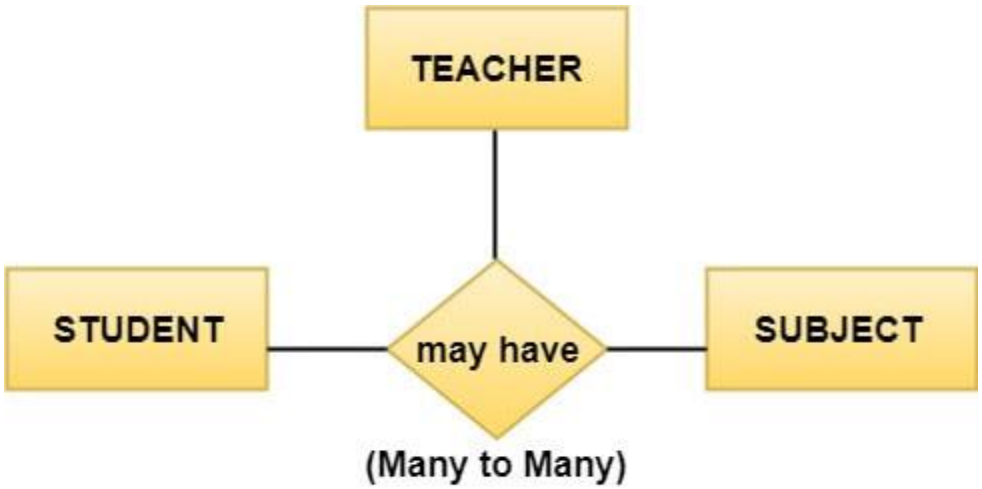


Fig: Ternary Relationship

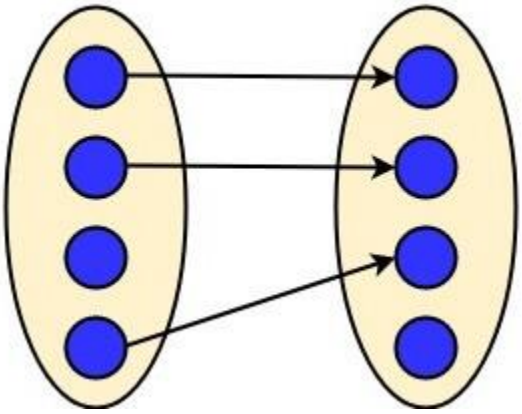
Cardinality

Cardinality describes the number of entities in one entity set, which can be associated with the number of entities of other sets via relationship set.

Types of Cardinalities

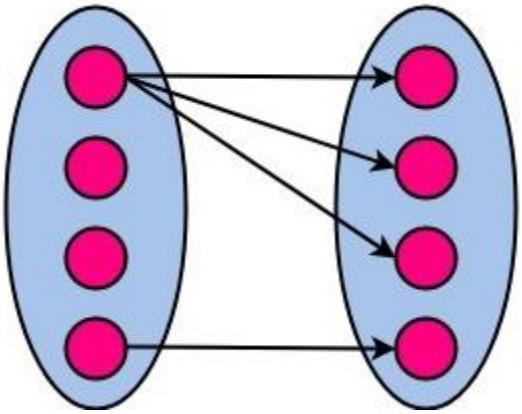
1. One to One: One entity from entity set A can be contained with at most one entity of entity set B and vice versa. Let us assume that each student has only one student ID, and each student ID is assigned to only one person. So, the relationship will be one to one.

Using Sets, it can be represented as:



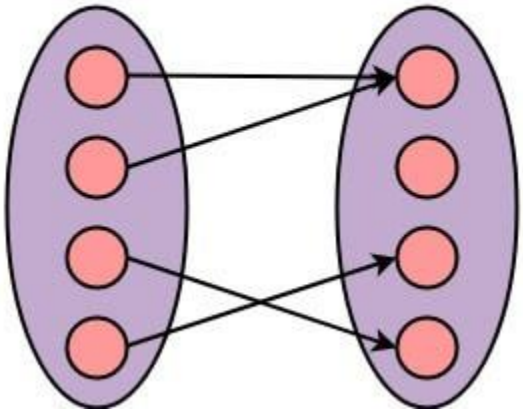
2. One to many: When a single instance of an entity is associated with more than one instances of another entity then it is called one to many relationships. For example, a client can place many orders; a order cannot be placed by many customers.

Using Sets, it can be represented as:



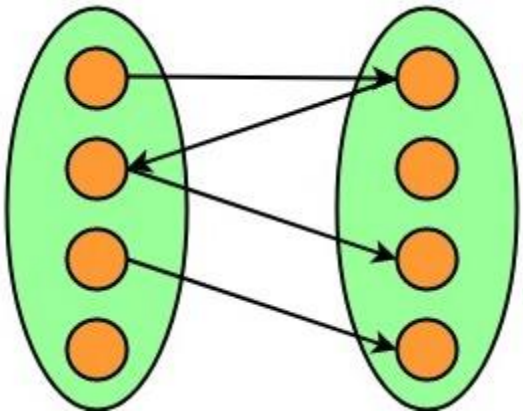
3. Many to One: More than one entity from entity set A can be associated with at most one entity of entity set B, however an entity from entity set B can be associated with more than one entity from entity set A. For example - many students can study in a single college, but a student cannot study in many colleges at the same time.

Using Sets, it can be represented as:



4. Many to Many: One entity from A can be associated with more than one entity from B and vice-versa. For example, the student can be assigned to many projects, and a project can be assigned to many students.

Using Sets, it can be represented as:

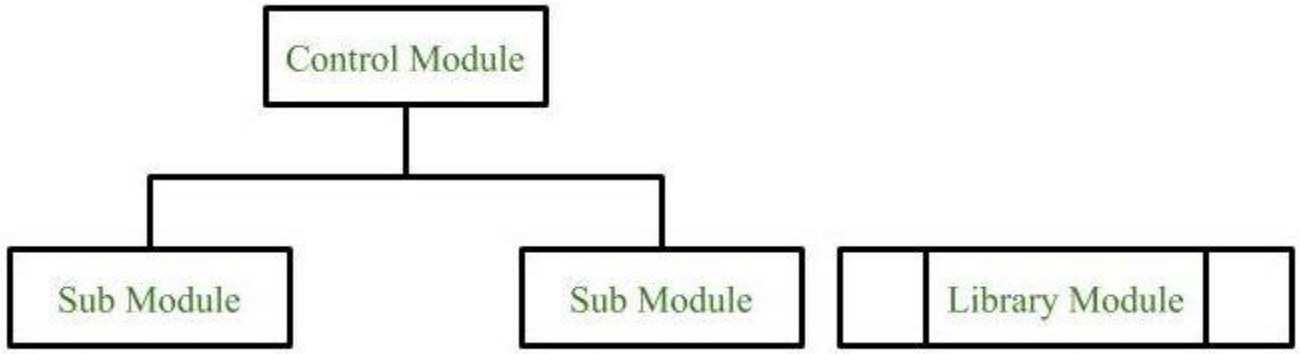


5.3 Structure Chart

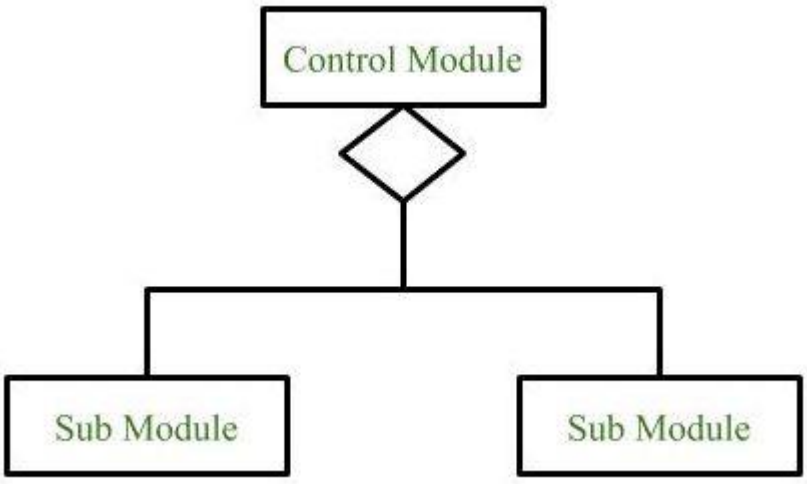
Structure Chart represent hierarchical structure of modules. It breaks down the entire system into lowest functional modules, describe functions and sub-functions of each module of a system to a greater detail. Structure Chart partitions the system into black boxes (functionality of the system is known to the users but inner details are unknown). Inputs are given to the black boxes and appropriate outputs are generated.
Modules at top level called modules at low level. Components are read from top to bottom and left to right. When a module calls another, it views the called module as black box, passing required parameters and receiving results.

Symbols used in construction of structured chart:-

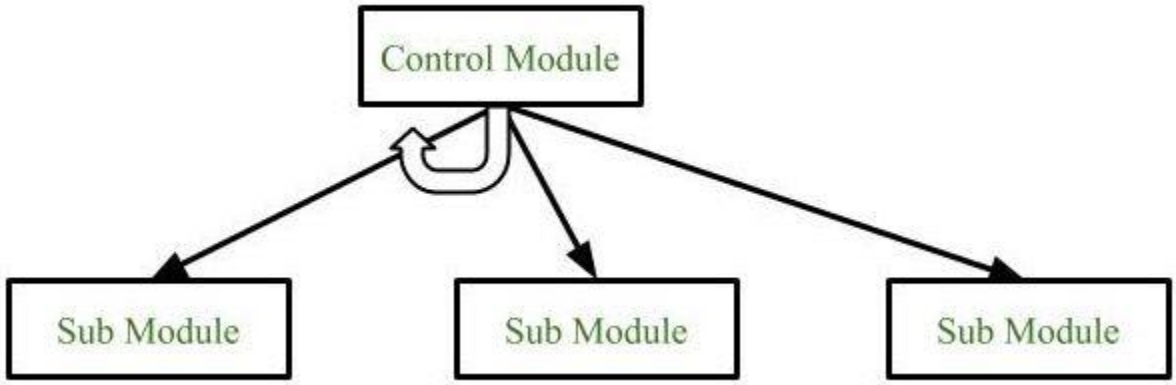
1. **Module**
   It represents the process or task of the system. It is of three types.
   - **Control Module**
     A control module branches to more than one sub module.
   - **Sub Module**
     Sub Module is a module which is the part (Child) of another module.
   - **Library Module**
     Library Module are reusable and invokable from any module.



2. **Conditional Call**
   It represents that control module can select any of the sub module on the basis of some condition.

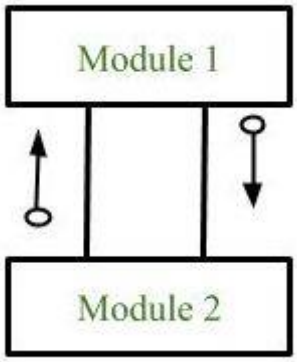


3. **Loop (Repetitive call of module)**
   It represents the repetitive execution of module by the sub module.
   A curved arrow represents loop in the module.

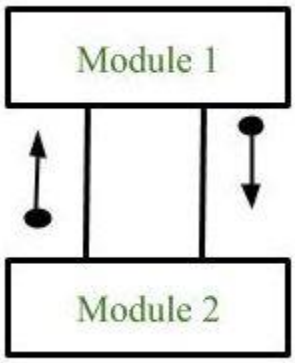


   All the sub modules cover by the loop repeat execution of module.

4. **Data Flow**
   It represents the flow of data between the modules. It is represented by directed arrow with empty circle at the end.

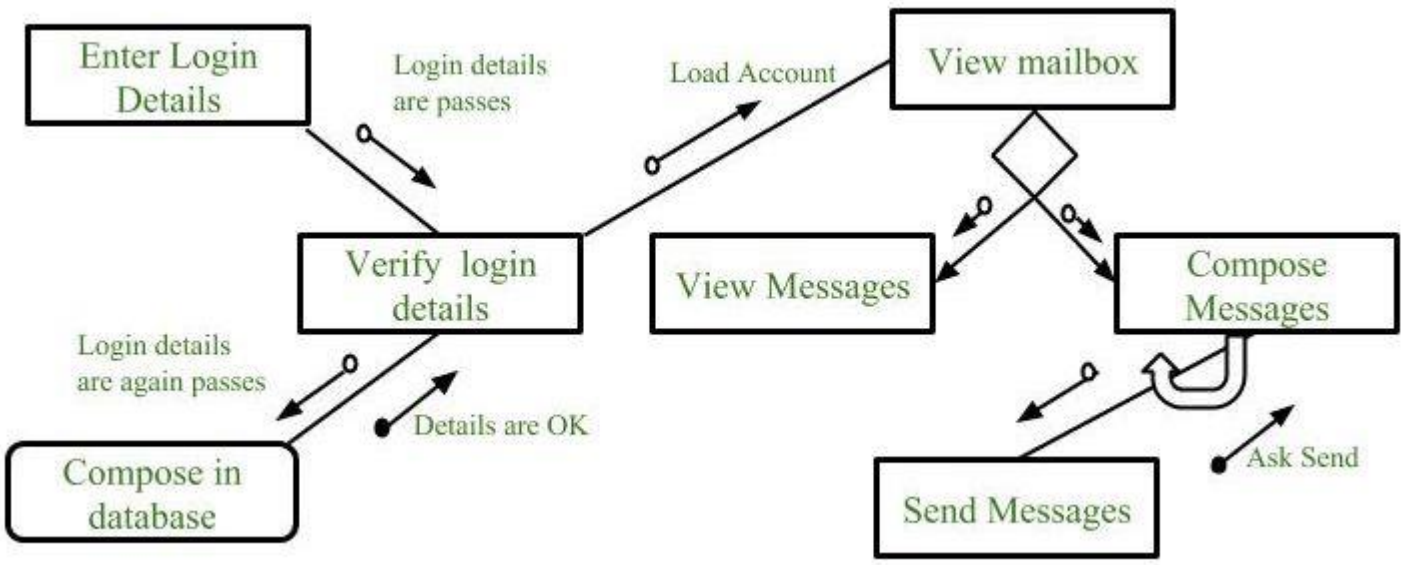


5. **Control Flow**
   It represents the flow of control between the modules. It is represented by directed arrow with filled circle at the end.



6. **Physical Storage**
   Physical Storage is that where all the information are to be stored.

Example : Structure chart for an Email server



Types of Structure Chart:

1. **Transform Centered Structure:**
   These type of structure chart are designed for the systems that receives an input which is transformed by a sequence of operations being carried out by one module.
2. **Transaction Centered Structure:**
   These structure describes a system that processes a number of different types of transaction.

5.3 Decision Table

A decision table is a brief visual representation for specifying which actions to perform depending on given conditions. The information represented in decision tables can also be represented as decision trees or in a programming language using if-then-else and switch-case statements.

A decision table is a good way to settle with different combination inputs with their corresponding outputs and is also called a cause-effect table. The reason to call cause-effect table is a related logical diagramming technique called cause-effect graphing that is basically used to obtain the decision table.

Importance of Decision Table:

- Decision tables are very much helpful in test design techniques.
- It helps testers to search the effects of combinations of different inputs and other software states that must correctly implement business rules.
- It provides a regular way of starting complex business rules, that is helpful for developers as well as for testers.

- It assists in the development process with the developer to do a better job. Testing with all combinations might be impractical.
- A decision table is basically an outstanding technique used in both testing and requirements management.
- It is a structured exercise to prepare requirements when dealing with complex business rules.
- It is also used in model complicated logic.

Decision Table in test designing:-

Blank Decision Table

CONDITIONS	STEP 1	STEP 2	STEP 3	STEP 4
Condition 1				
Condition 2				
Condition 3				
Condition 4				

Decision Table: Combinations

CONDITIONS	STEP 1	STEP 2	STEP 3	STEP 4
Condition 1	Y	Y	N	N
Condition 2	Y	N	Y	N
Condition 3	Y	N	N	Y
Condition 4	N	Y	Y	N

Advantages of Decision Table :-

- Any complex business flow can be easily converted into test scenarios & test cases using this technique.
- Decision tables work iteratively which means the table created at the first iteration is used as input tables for the next tables. The iteration is done only if the initial table is not satisfactory.
- Simple to understand and everyone can use this method to design the test scenarios & test cases.
- It provides complete coverage of test cases which helps to reduce the rework on writing test scenarios & test cases.
- These tables guarantee that we consider every possible combination of condition values. This is known as its completeness property.

5.4 Decision Tree

A **Decision Tree** offers a graphic read of the processing logic concerned in a higher cognitive process and therefore the corresponding actions are taken. The perimeters of a choice tree represent conditions and therefore the leaf nodes represent the actions to be performed looking at the result of testing the condition.
For example, consider Library Membership Automation Software (LMS) where it ought to support the following three options: New member, Renewal, and Cancel membership. These are explained as following below.

1. New Member Option:

- **Decision:**
  Once the 'new member' possibility is chosen, the software system asks for details concerning the member just like the member's name, address, number, etc.
- **Action:**
  If correct info is entered then a membership record for the member is made and a bill is written for the annual membership charge and the protection deposit collectible.

Renewal Option:

- **Decision:**
  If the 'renewal' possibility is chosen, the LMS asks for the member's name and his membership range to test whether or not he's a sound member or not.

- **Action:**
  If the membership is valid then the membership ending date is updated and therefore the annual membership bill is written, otherwise, a slip-up message is displayed.

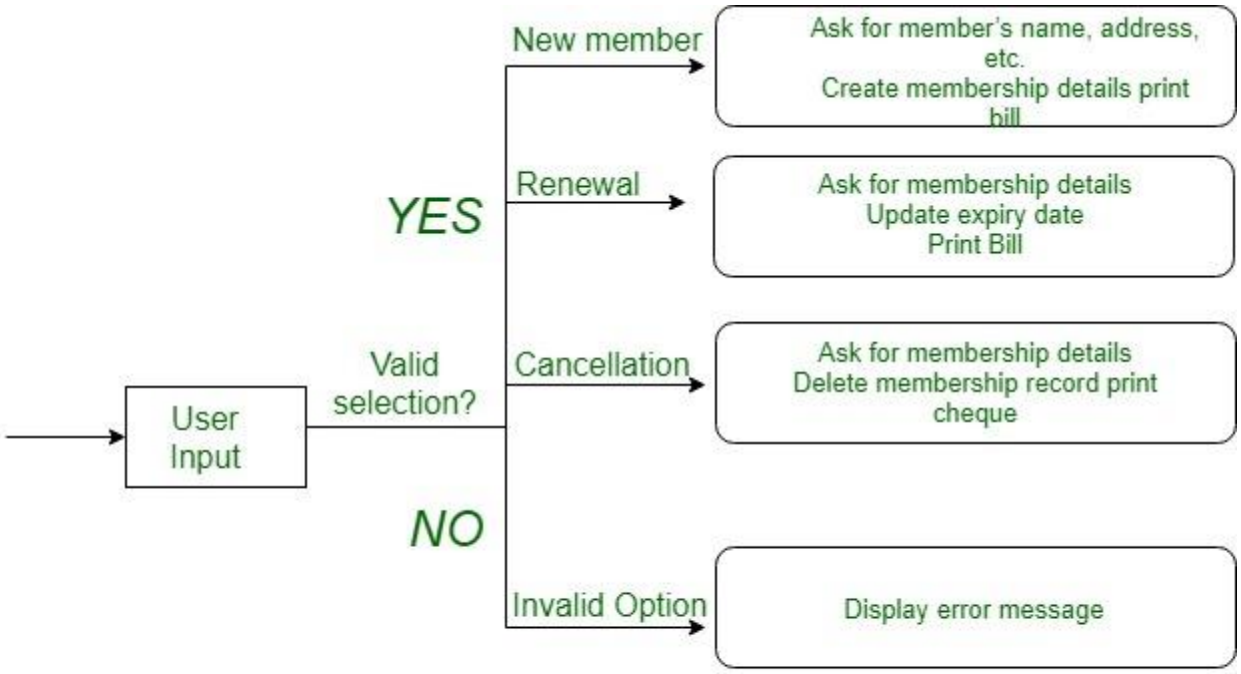
Cancel Membership Option:

- **Decision:**
  If the 'cancel membership' possibility is chosen, then the software system asks for a member's name and his membership range.

- **Action:**
  The membership is off, a cheque for the balance quantity because of the member is written and at last, the membership record is deleted from the information.

Decision tree representation of the above example:

The following tree shows the graphical illustration of the above example, when obtaining data from the user, the system makes a choice and then performs the corresponding actions.



Decision tree for LMS

5.5 Use case Diagram

What is the Use Case Diagram?

Use Case Diagram captures the system's functionality and requirements by using actors and use cases. Use Cases model the services, tasks, function that a system needs to perform. Use cases represent high-level functionalities and how a user will handle the system. Use-cases are the core concepts of Unified Modelling language modeling.

Why Use-Case diagram?

A Use Case consists of use cases, persons, or various things that are invoking the features called as actors and the elements that are responsible for implementing the use cases. Use case diagrams capture the dynamic behaviour of a live system. It models how an external entity interacts with the system to make it work. Use

case diagrams are responsible for visualizing the external things that interact with the part of the system.

Use-case diagram notations

Following are the common notations used in a use case diagram:

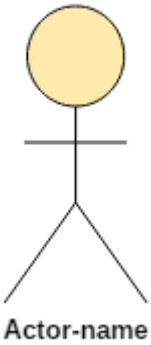
Use-case:

Use cases are used to represent high-level functionalities and how the user will handle the system. A use case represents a distinct functionality of a system, a component, a package, or a class. It is denoted by an oval shape with the name of a use case written inside the oval shape. The notation of a use case in UML is given below:

UML UseCase Notation

Actor:

It is used inside use case diagrams. The actor is an entity that interacts with the system. A user is the best example of an actor. An actor is an entity that initiates the use case from outside the scope of a use case. It can be any element that can trigger an interaction with the use case. One actor can be associated with multiple use cases in the system. The actor notation in UML is given below.



UML Actor Notation

How to draw a use-case diagram?

To draw a use case diagram in UML first one need to analyse the entire system carefully. You have to find out every single function that is provided by the system. After all the functionalities of a system are found out, then these functionalities are converted into various use cases which will be used in the use case diagram.

A use case is nothing but a core functionality of any working system. After organizing the use cases, we have to enlist the various actors or things that are going to interact with the system. These actors are responsible for invoking the functionality of a system. Actors can be a person or a thing. It can also be a private entity of a system. These actors must be relevant to the functionality or a system they are interacting with.

After the actors and use cases are enlisted, then you have to explore the relationship of a particular actor with the use case or a system. One must identify the total number of ways an actor could interact with the system. A single actor can interact with multiple use cases at the same time, or it can interact with numerous use cases simultaneously.

Following rules must be followed while drawing use-case for any system:

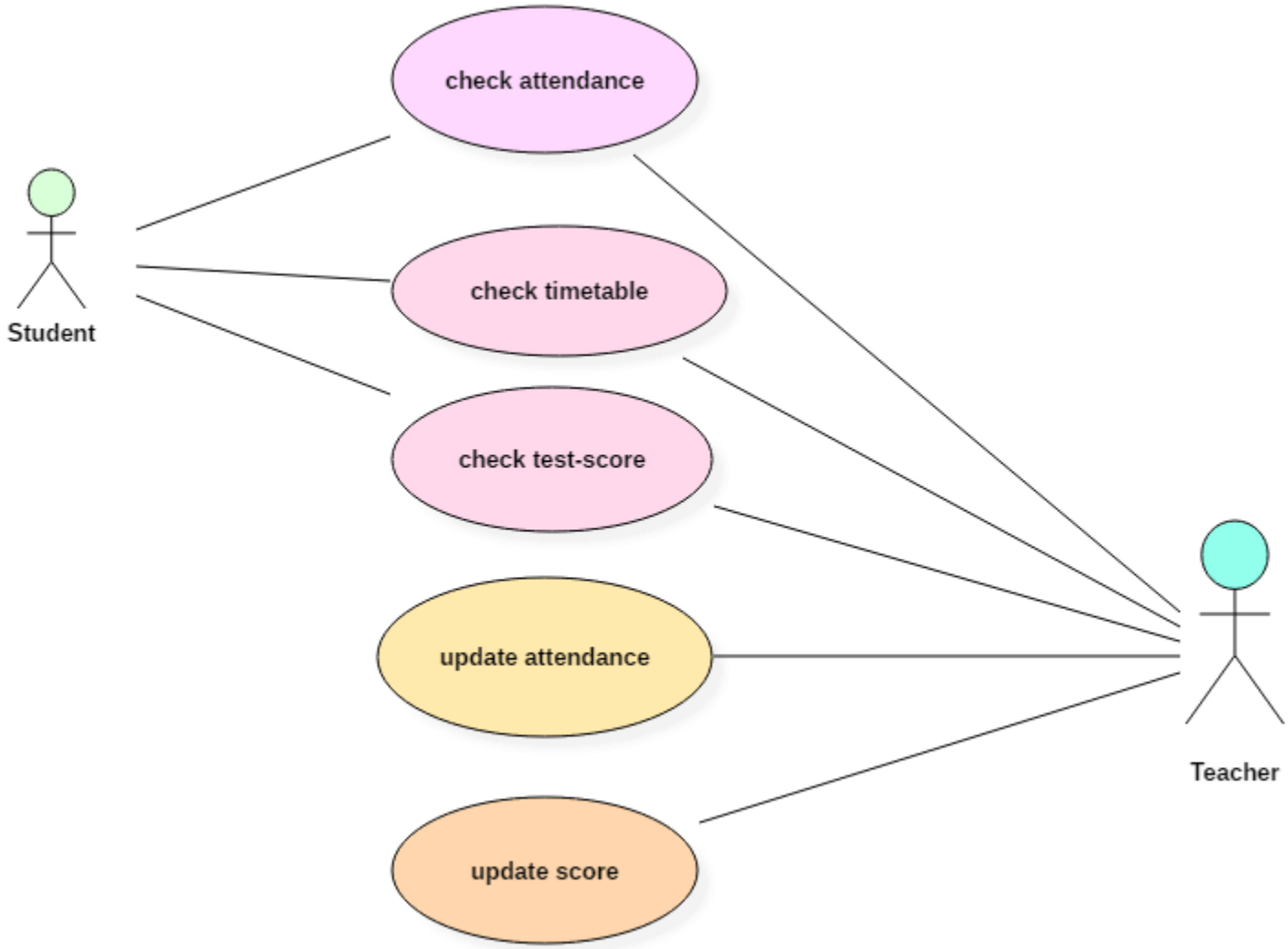
1. The name of an actor or a use case must be meaningful and relevant to the system.
2. Interaction of an actor with the use case must be defined clearly and in an understandable way.
3. Annotations must be used wherever they are required.
4. If a use case or an actor has multiple relationships, then only significant interactions must be displayed.

Tips for drawing a use-case diagram

1. A use case diagram should be as simple as possible.
2. A use case diagram should be complete.
3. A use case diagram should represent all interactions with the use case.
4. If there are too many use cases or actors, then only the essential use cases should be represented.
5. A use case diagram should describe at least a single module of a system.
6. If the use case diagram is large, then it should be generalized.

An example of a use-case diagram

Following use case diagram represents the working of the student management system:



UML UseCase Diagram

In the above use case diagram, there are two actors named student and a teacher. There are a total of five use cases that represent the specific functionality of a student management system. Each actor interacts with a particular use case. A student actor can check attendance, timetable as well as test marks on the application or a system. This actor can perform only these interactions with the system even though other use cases are remaining in the system.

It is not necessary that each actor should interact with all the use cases, but it can happen.

The second actor named teacher can interact with all the functionalities or use cases of the system. This actor can also update the attendance of a student and marks of the student. These interactions of both student and a teacher actor together sums up the entire student management application.

When to use a use-case diagram?

A use case is a unique functionality of a system which is accomplished by a user. A purpose of use case diagram is to capture core functionalities of a system and visualize the interactions of various things called as actors with the use case. This is the general use of a use case diagram.

The use case diagrams represent the core parts of a system and the workflow between them. In use case, implementation details are hidden from the external use only the event flow is represented.

With the help of use case diagrams, we can find out pre and post conditions after the interaction with the actor. These conditions can be determined using various test cases.

In general use case diagrams are used for:

1. Analyzing the requirements of a system
2. High-level visual software designing
3. Capturing the functionalities of a system
4. Modeling the basic idea behind the system
5. Forward and reverse engineering of a system using various test cases.

Use cases are intended to convey desired functionality so the exact scope of a use case may vary according to the system and the purpose of creating UML model.

5.6 Sequence Diagram

Sequence Diagram

The sequence diagram represents the flow of messages in the system and is also termed as an event diagram. It helps in envisioning several dynamic scenarios. It portrays the communication between any two lifelines as a time-ordered sequence of events, such that these lifelines took part at the run time. In UML, the lifeline is represented by a vertical bar, whereas the message flow is represented by a vertical dotted line that extends across the bottom of the page. It incorporates the iterations as well as branching.

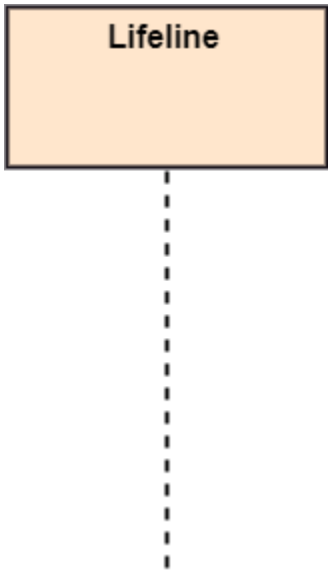
Purpose of a Sequence Diagram

1. To model high-level interaction among active objects within a system.
2. To model interaction among objects inside a collaboration realizing a use case.
3. It either models generic interactions or some certain instances of interaction.

Notations of a Sequence Diagram

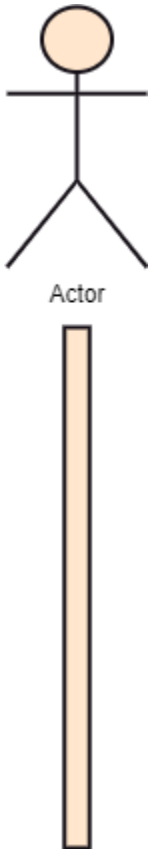
Lifeline

An individual participant in the sequence diagram is represented by a lifeline. It is positioned at the top of the diagram.



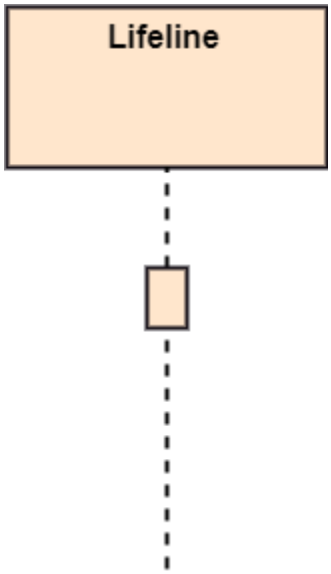
Actor

A role played by an entity that interacts with the subject is called as an actor. It is out of the scope of the system. It represents the role, which involves human users and external hardware or subjects. An actor may or may not represent a physical entity, but it purely depicts the role of an entity. Several distinct roles can be played by an actor or vice versa.



Activation

It is represented by a thin rectangle on the lifeline. It describes that time period in which an operation is performed by an element, such that the top and the bottom of the rectangle is associated with the initiation and the completion time, each respectively.

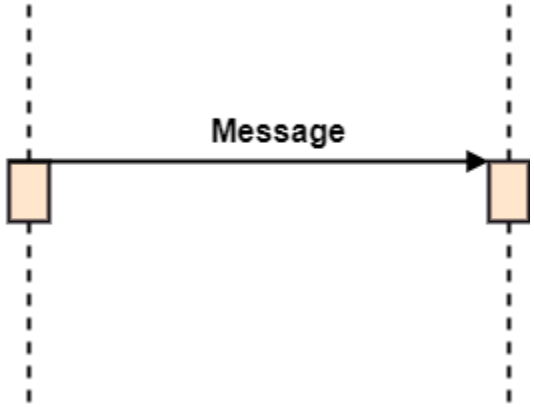


Messages

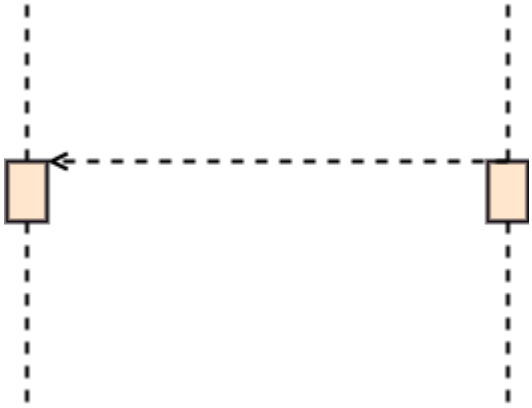
The messages depict the interaction between the objects and are represented by arrows. They are in the sequential order on the lifeline. The core of the sequence diagram is formed by messages and lifelines.

Following are types of messages enlisted below:

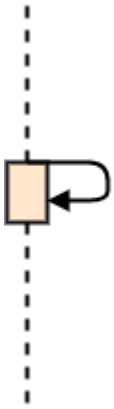
- **Call Message:** It defines a particular communication between the lifelines of an interaction, which represents that the target lifeline has invoked an operation.



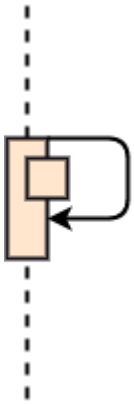
- **Return Message:** It defines a particular communication between the lifelines of interaction that represent the flow of information from the receiver of the corresponding caller message.



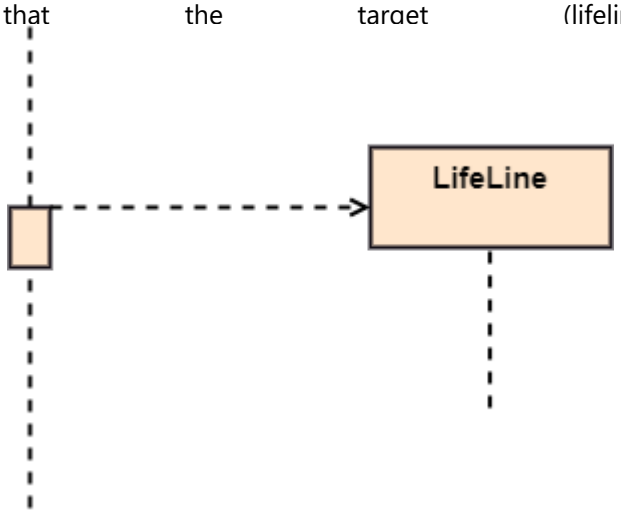
- **Self Message:** It describes a communication, particularly between the lifelines of an interaction that represents a message of the same lifeline, has been invoked.



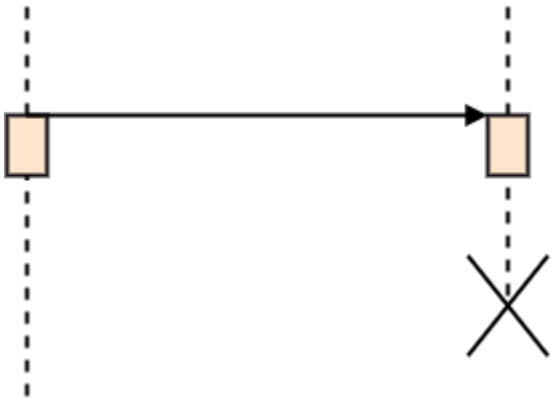
- **Recursive Message:** A self message sent for recursive purpose is called a recursive message. In other words, it can be said that the recursive message is a special case of the self message as it represents the recursive calls.



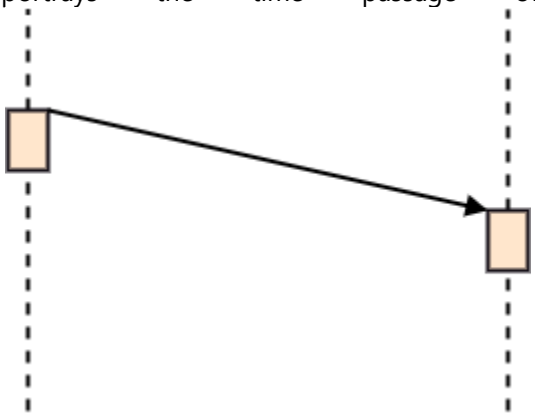
- **Create Message:** It describes a communication, particularly between the lifelines of an interaction describing that the target (lifeline) has been instantiated.



- **Destroy Message:** It describes a communication, particularly between the lifelines of an interaction that depicts a request to destroy the lifecycle of the target.



- **Duration Message:** It describes a communication particularly between the lifelines of an interaction, which portrays the time passage of the message while modeling a system.



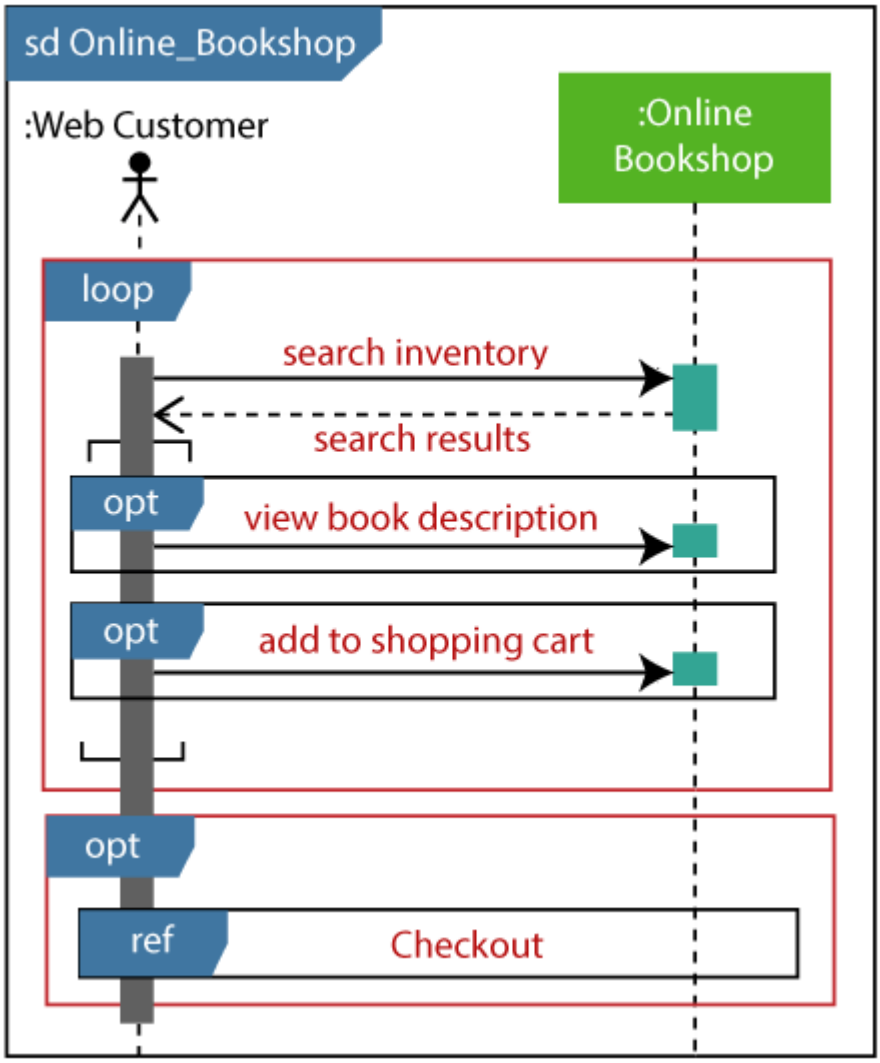
Note

A note is the capability of attaching several remarks to the element. It basically carries useful information for the modelers.

Example of a Sequence Diagram

An example of a high-level sequence diagram for online bookshop is given below.

Any online customer can search for a book catalog, view a description of a particular book, add a book to its shopping cart, and do checkout.



Benefits of a Sequence Diagram

1. It explores the real-time application.
2. It depicts the message flow between the different objects.
3. It has easy maintenance.
4. It is easy to generate.
5. Implement both forward and reverse engineering.
6. It can easily update as per the new change in the system.

The drawback of a Sequence Diagram

1. In the case of too many lifelines, the sequence diagram can get more complex.

2. The incorrect result may be produced, if the order of the flow of messages changes.
3. Since each sequence needs distinct notations for its representation, it may make the diagram more complex.
4. The type of sequence is decided by the type of message.

Unit - 6 (Project Work)

6.1 Web page Development
6.2 Game development
6.3 Mobile Application development
6.4 Software piracy protection
6.5 E-learning platform

Note :- *In this unit you need to create some projects and apply all those methodologies , techniques which we have studied so far in software engineering and also need to prepare a proper documentation for your project (a project report for each project).*

But for your reference some projects are given through which you can practice and learn.

Web page development (Create a web page for your school by taking reference from below code)

Note:- This is just an example for your idea you need to create a web page for your school by yourself.

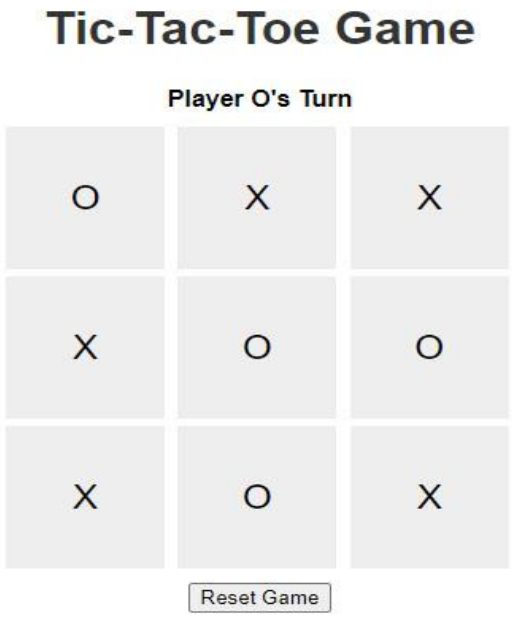
```
<!DOCTYPE html>
<html>
<head>
   <title>Shree Janta Model School</title>
   <style>
      body {
         font-family: Arial, sans-serif;
         text-align: center;
         background-color: #f0f0f0;
height="300">
      }
```

```
        header {
            background-color: #006db6;
            color: #fff;
            padding: 20px;
        }
        h1 {
            margin: 0;
        }
        nav {
            background-color: #0099e5;
            color: #fff;
            padding: 10px;
        }
        .container {
            max-width: 800px;
            margin: 0 auto;
            padding: 20px;
        }
        h2 {
            color: #006db6;
        }
        p {
            line-height: 1.5;
        }
    </style>
</head>
<body>
    <header>
        <h1>Welcome to Shree Janta Model School</h1>
    </header>
<img src="images.jpg" alt="Shree Janta Model School" width="400"
height="300">
    <nav>
        <ul>
            <li><a href="#about">About Us</a></li>
            <li><a href="#classes">Classes</a></li>
            <li><a href="#contact">Contact Us</a></li>
        </ul>
    </nav>
    <div class="container">
```

```
        <section id="about">
            <h2>About Us</h2>
            <p>Welcome to Shree Janta Model School, Madai Ugreepatti. We are dedicated to providing quality education and fostering holistic development in our students. Our school offers a range of classes from 0 to 12, including diploma and pre-diploma programs. We are committed to excellence in education.</p>
        </section>
        <section id="classes">
            <h2>Classes Offered</h2>
            <p>We offer a comprehensive range of classes to meet the educational needs of our students:</p>
            <ul>
                <li>Class 0</li>
                <li>Class 1 to 12</li>
                <li>Diploma Classes</li>
                <li>Pre-Diploma Classes</li>
            </ul>
        </section>
        <section id="contact">
            <h2>Contact Us</h2>
            <p>If you have any questions or would like to get in touch, please feel free to contact us at:</p>
            <p>Email: info@shreejantaschool.com</p>
            <p>Phone: +123-456-7890</p>
        </section>
    </div>
</body>
</html>
```

6.2 Game Development and web page example too.

Example - 2 (Creating a Tic - Tac - Toe Game) using HTML , CSS and JavaScript.



This a complete example for web page development and game development also and same game code will be given in java also so enjoy and learn.

Html code - (Save it as index.html in a separate folder)

```
<!DOCTYPE html>
<html>
<head>
    <title>Tic-Tac-Toe Game</title>
    <link rel="stylesheet" type="text/css" href="style.css">
</head>
<body>
    <div class="container">
        <h1>Tic-Tac-Toe Game</h1>
        <div id="message">Player X's Turn</div>
        <div class="board" id="board">
            <div class="cell" id="0"></div>
            <div class="cell" id="1"></div>
            <div class="cell" id="2"></div>
```

```
            <div class="cell" id="3"></div>
            <div class="cell" id="4"></div>
            <div class="cell" id="5"></div>
            <div class="cell" id="6"></div>
            <div class="cell" id="7"></div>
            <div class="cell" id="8"></div>
        </div>
        <button id="reset-button">Reset Game</button>
    </div>
    <script src="script.js"></script>
</body>
</html>
```

CSS code (save it as style.css in the same html folder)

```
body {
    font-family: Arial, sans-serif;
    text-align: center;
}

.container {
    max-width: 320px;
    margin: 0 auto;
}

h1 {
    color: #333;
}

.board {
    display: grid;
    grid-template-columns: repeat(3, 1fr);
    gap: 5px;
    margin-top: 10px;
}

.cell {
    width: 100px;
```

```
    height: 100px;
    background-color: #eee;
    display: flex;
    justify-content: center;
    align-items: center;
    font-size: 24px;
    cursor: pointer;
}

#message {
    font-weight: bold;
    margin-top: 10px;
}

#reset-button {
    display: block;
    margin: 10px auto;
}
```

Javascript code - (save it as script.js in the same folder)

```
// JavaScript for Tic-Tac-Toe game

const board = document.getElementById('board');
const cells = document.querySelectorAll('.cell');
const message = document.getElementById('message');
const resetButton = document.getElementById('reset-button');

let currentPlayer = 'X';
let gameActive = true;

const winningCombos = [
    [0, 1, 2],
    [3, 4, 5],
    [6, 7, 8],
    [0, 3, 6],
    [1, 4, 7],
    [2, 5, 8],
    [0, 4, 8],
    [2, 4, 6],
```

```
];

const checkWinner = () => {
    for (let combo of winningCombos) {
        const [a, b, c] = combo;
        if (cells[a].textContent && cells[a].textContent === cells[b].textContent
&& cells[a].textContent === cells[c].textContent) {
            gameActive = false;
            message.textContent = `Player ${currentPlayer} wins!`;
            cells[a].classList.add('win');
            cells[b].classList.add('win');
            cells[c].classList.add('win');
        }
    }

    if (!gameActive) return;

    if ([...cells].every(cell => cell.textContent)) {
        gameActive = false;
        message.textContent = "It's a draw!";
    }
};

const handleCellClick = (event) => {
    const cell = event.target;
    const cellIndex = cell.id;

    if (cell.textContent || !gameActive) return;

    cell.textContent = currentPlayer;
    cell.classList.add(currentPlayer);

    checkWinner();

    currentPlayer = currentPlayer === 'X' ? 'O' : 'X';
    message.textContent = `Player ${currentPlayer}'s Turn`;
};

const handleResetClick = () => {
    cells.forEach(cell => {
        cell.textContent = '';
```

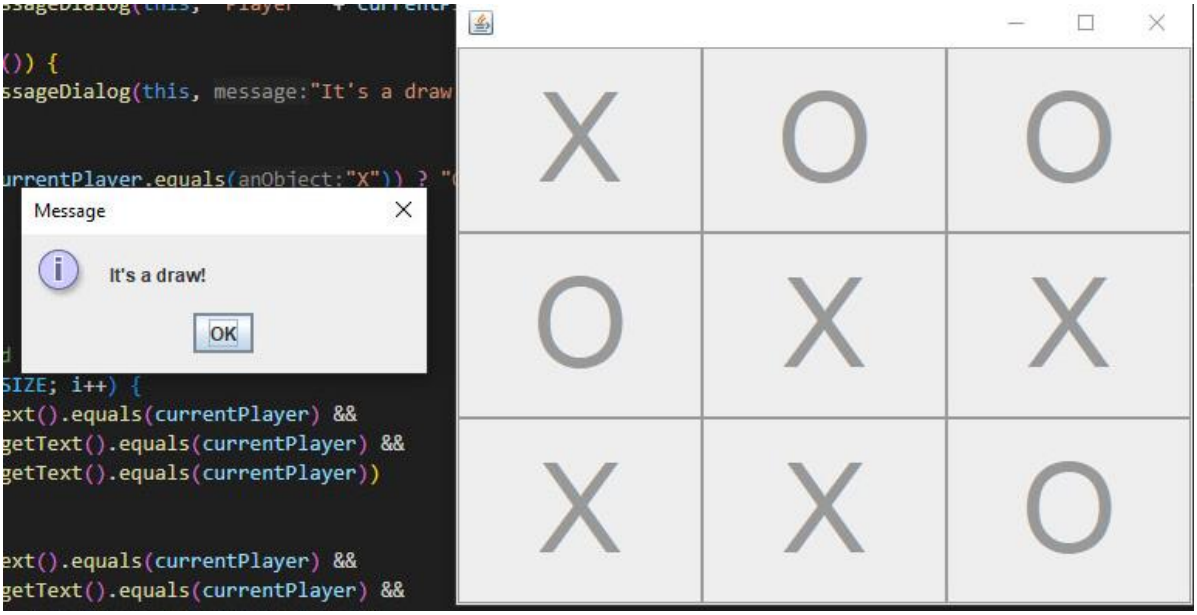
```
        cell.classList.remove('X', 'O', 'win');
    });

    currentPlayer = 'X';
    gameActive = true;
    message.textContent = "Player X's Turn";
};

cells.forEach(cell => cell.addEventListener('click', handleCellClick));
resetButton.addEventListener('click', handleResetClick);
```

Now the same game code in java

Save the file as TicTacToeGame.java in any folder and run it through visual studio code (IDE).



Code:-

```java
import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;

public class TicTacToeGame extends JFrame implements ActionListener {
    private static final int BOARD_SIZE = 3;
    private JButton[][] buttons;
    private String currentPlayer;

    public TicTacToeGame() {
        buttons = new JButton[BOARD_SIZE][BOARD_SIZE];
        currentPlayer = "X";
```

```
        // Initialize the game board
        for (int i = 0; i < BOARD_SIZE; i++) {
            for (int j = 0; j < BOARD_SIZE; j++) {
                buttons[i][j] = new JButton("");
                buttons[i][j].setFont(new Font("Arial", Font.PLAIN, 80));
                buttons[i][j].addActionListener(this);
                add(buttons[i][j]);
            }
        }

        setLayout(new GridLayout(BOARD_SIZE, BOARD_SIZE));
        setSize(300, 300);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setVisible(true);
    }

    public void actionPerformed(ActionEvent e) {
        JButton button = (JButton) e.getSource();

        if (button.getText().equals("")) {
            button.setText(currentPlayer);
            button.setEnabled(false);
            if (checkWin()) {
                JOptionPane.showMessageDialog(this, "Player " + currentPlayer + " wins!");
                resetBoard();
            } else if (isBoardFull()) {
                JOptionPane.showMessageDialog(this, "It's a draw!");
                resetBoard();
            } else {
                currentPlayer = (currentPlayer.equals("X")) ? "O" : "X";
            }
        }
    }

    private boolean checkWin() {
        // Check rows, columns, and diagonals for a win
        for (int i = 0; i < BOARD_SIZE; i++) {
            if (buttons[i][0].getText().equals(currentPlayer) &&
                    buttons[i][1].getText().equals(currentPlayer) &&
                    buttons[i][2].getText().equals(currentPlayer))
                return true;

            if (buttons[0][i].getText().equals(currentPlayer) &&
                    buttons[1][i].getText().equals(currentPlayer) &&
                    buttons[2][i].getText().equals(currentPlayer))
                return true;
        }

        if (buttons[0][0].getText().equals(currentPlayer) &&
                buttons[1][1].getText().equals(currentPlayer) &&
                buttons[2][2].getText().equals(currentPlayer))
            return true;

        if (buttons[0][2].getText().equals(currentPlayer) &&
                buttons[1][1].getText().equals(currentPlayer) &&
                buttons[2][0].getText().equals(currentPlayer))
            return true;

        return false;
    }

    private boolean isBoardFull() {
        for (int i = 0; i < BOARD_SIZE; i++) {
            for (int j = 0; j < BOARD_SIZE; j++) {
```

```
                if (buttons[i][j].getText().equals("")) {
                    return false;
                }
            }
        }
        return true;
    }

    private void resetBoard() {
        for (int i = 0; i < BOARD_SIZE; i++) {
            for (int j = 0; j < BOARD_SIZE; j++) {
                buttons[i][j].setText("");
                buttons[i][j].setEnabled(true);
            }
        }
        currentPlayer = "X";
    }

    public static void main(String[] args) {
        new TicTacToeGame();
    }
}
```

6.3 Mobile Application development

Developing a mobile application involves several steps, including planning, design, development, testing, and deployment. Here's an overview of the process and a simple example of creating a mobile application using the Android platform:

Steps to Develop a Mobile Application:

1. Idea and Concept:

Start with a clear idea of the mobile app's purpose and target audience.
Research the market and identify potential competitors.

2. Planning:

Define the app's features, functionality, and scope.
Create wireframes or prototypes to visualize the app's layout.
Plan the user experience (UX) and user interface (UI) design.

3. Technology Stack:

Choose the platform for development (e.g., Android, iOS, or cross-platform).
Select the programming languages, development tools, and frameworks.

4. Design:

Create the app's visual design, including icons, images, and branding.
Design the app's user interface for a seamless user experience.

5. Development:

Write the app's code based on the chosen technology stack.
Implement the planned features and functionality.
Test the app as you develop to catch and fix issues early.

6. Testing:

Perform thorough testing to identify and resolve bugs and issues.
Test the app on different devices and screen sizes.
Conduct usability testing for the app's overall user experience.

7. Optimization:

Optimize the app's performance, including speed and responsiveness.
Address security and privacy concerns, if applicable.
Ensure the app is compatible with the latest operating system versions.

8. Deployment:

Prepare the app for submission to app stores (e.g., Google Play Store for Android).
Follow the app store guidelines for listing and publishing the app.

9. Marketing and Promotion:

Develop a marketing strategy to promote the app.
Use various channels, including social media, email marketing, and paid advertising.

10. Maintenance and Updates:

Regularly update the app to add new features and fix issues.
Gather user feedback and make improvements based on user suggestions.

Creating a Simple Android Mobile Application (Hello World):

Please Note:- *This is just a demonstration to have some ideas ; android studio is a heavy software so it's not compulsory to run the code and create but to have understanding this is given.*

Here's a basic example of creating a "Hello World" mobile application for Android using Java:

1. Install Android Studio:

Download and install Android Studio, the official integrated development environment for Android app development.

2. Create a New Project:

Open Android Studio and select "Start a new Android Studio project."
Follow the on-screen instructions to set up your project.

3. Design the User Interface:

In the "res" folder, you'll find the "layout" folder.
Open "activity_main.xml" and design your app's user interface.

4. Add Code:

In the "java" folder, you'll find the main activity file.

Open the "MainActivity.java" file and add the following code:

```
package com.example.myfirstapp;
import androidx.appcompat.app.AppCompatActivity;
import android.os.Bundle;

public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
```

```
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }
}
```

5. Run the App:

- Connect an Android device or use an emulator.
- Click the "Run" button in Android Studio to run the app.

This simple "Hello World" app will display a basic user interface with the text "Hello World." You can customize and add more features to your app as you become more familiar with Android app development.

Keep in mind that mobile app development involves more complexity when creating real-world applications, but this basic example should help you get started with Android app development. For iOS app development, you would use Xcode and Swift or Objective-C as the programming languages.

6.4 Software piracy protection

Software piracy protection, also known as software license protection, is the process of safeguarding software applications from unauthorized distribution, copying, or use. Unauthorized software piracy can lead to financial losses for software developers and can compromise the security and integrity of their products. Here are common methods and strategies to protect software from piracy:

1. **License Keys and Serial Numbers:**

- Require users to enter a unique license key or serial number during installation.
- Verify the validity of the key with a remote server.
- Limit the number of installations allowed per license key.

2. Online Activation:

- Require users to activate the software online, which validates their license.
- Software checks with a server to ensure the license is genuine.
- This method allows for additional security measures, such as deactivation if the license is used on multiple devices.

3. **Hardware Locking:**

- Tie the software to specific hardware components, such as a unique hardware ID.

- The software runs only on the authorized hardware.
- This method is commonly used in dongles, which are physical hardware devices connected to the computer.

4. Trial Versions:

- Offer a free trial version with limited features or a time limit.
- Encourage users to purchase a full license for unrestricted access.
- Restrict the functionality in the trial version.

5. Obfuscation and Code Protection:

- Apply code obfuscation techniques to make reverse engineering difficult.
- Protect sensitive parts of the code with encryption.
- Monitor the software for tampering or debugging attempts.

6. Digital Rights Management (DRM):

- Use DRM technology to manage and control access to digital content.
- DRM systems prevent unauthorized copying and distribution of digital media, including software.

7. Server-Based Licensing:

- Host the core functionality of the software on a server.
- Users can access the software through a remote connection and pay for usage.
- This method is common in Software as a Service (SaaS) applications.

8. Frequent Updates and Patches:

- Release frequent software updates and patches.
- Make it challenging for pirated versions to stay current.
- Encourage legitimate users to stay up to date by offering new features and improvements.

9. Legal Protections:

- Register your software and enforce intellectual property rights through legal means.
- Pursue legal action against software pirates when necessary.

10. User Education and Licensing Agreements:

- Clearly communicate licensing terms and conditions to users.
- Educate users about the importance of purchasing legitimate software.
- Include legal clauses that restrict unauthorized distribution in licensing agreements.

11. Watermarking:

- Embed watermarks in documents or media created with the software.
- Watermarks deter users from sharing documents or content created using pirated software.

12. Product Activation and Registration:

- Require users to activate and register the software after purchase.
- Gather user data to track and verify the authenticity of software installations.

6.5 E-learning platforms

An e-learning platform, also known as a learning management system (LMS), is a digital technology or software solution designed to facilitate online learning and education. These platforms provide a virtual environment where students and teachers can engage in various educational activities, such as accessing and sharing course materials, participating in discussions, taking quizzes, submitting assignments, and more. E-learning platforms are commonly used for both formal education (in schools and universities) and informal, lifelong learning.

Some well-known e-learning platforms globally include:

1. Moodle: An open-source learning platform widely used in educational institutions. It offers a range of features for creating and delivering online courses.

2. Canvas by Instructure: A popular LMS used by many universities and K-12 schools, known for its user-friendly interface.

3. Blackboard Learn: A well-established LMS with a range of features for course management, content delivery, and communication.

4. edX: A massive open online course (MOOC) platform that offers a wide range of courses from universities and institutions worldwide.

5. Coursera: Another leading MOOC platform that partners with top universities to offer a diverse set of courses.

6. Udemy: A platform for instructors to create and sell their online courses on a variety of subjects.

7. Khan Academy: A nonprofit platform providing free educational content in a wide range of subjects, particularly focused on K-12 education.

8. LinkedIn Learning (formerly Lynda.com): Offers a vast library of courses on professional skills, software, and business topics.

9. Google Classroom: Designed for K-12 and higher education, it integrates with Google's suite of educational tools and services.

10. Schoology: A K-12 LMS that provides a collaborative platform for teachers, students, and administrators.

In Nepal, e-learning platforms have also gained prominence, especially in the wake of the COVID-19 pandemic when online education became a necessity. Some e-learning platforms in Nepal include:

1) Nepal Open University (NOU): Offers various online courses, particularly for higher education and adult learners.

2) edusanjal.com: Provides a platform for students to access information about colleges, courses, and scholarships.

3) Kullabs: Offers a collection of digital learning resources, including videos, notes, and quizzes, for students in Nepal.

4) HamroPathshala: An e-learning platform that provides video lectures and course materials for school-level education.

5) NPTEL (Nepal): An initiative by the Ministry of Education, Science, and Technology of Nepal, it offers free online courses in collaboration with Indian universities.